

28.04.2016

Удовлетворенность работой в командах по гибкой разработке Agile DevOps как перепроектирование работы

Джон Ф. Трипп

Бэйлор Университет, john_tripp@baylor.edu

Синди Рименшнайдер ,

Бэйлор Университет, C_Riemenschneider@baylor.edu

Джейсон Б. Тэтчер ,

Университет Клемсона, jason.b.thatcher@gmail.com

Следите за этим и другими работами по адресу: <https://aisel.aisnet.org/jais>

Рекомендуемая литература

Трипп, Джон Ф.; Рименшнайдер, Синди; и Тэтчер, Джейсон Б. (2016) «Удовлетворенность работой в командах гибкой разработки: гибкая разработка как перепроектирование Системы работы», , 17(4), .

DOI: 10.17705/1jais.00426

Доступно по адресу: <https://aisel.aisnet.org/jais/vol17/iss4/1>

Этот материал предоставлен вам журналами AIS в электронной библиотеке AIS (AISeL). Он был принят к публикации в журнале Ассоциации информационных систем уполномоченным администратором электронной библиотеки AIS (AISeL). Для получения дополнительной информации обращайтесь по адресу elibrary@aisnet.org.



Удовлетворенность работой в командах Agile-разработки: Agile Развитие как перепроектирование работы

Джон Ф. Трипп

Кафедра информационных систем, Университет Бэйлора
John_Tripp@baylor.edu

Синди Рименшнайдер

Кафедра информационных систем, Университет Бэйлора
C_Riemenschneider@baylor.edu

Джейсон Б. Тэтчер

Факультет менеджмента, Университет Клемсона
jason.b.thatcher@gmail.com

Абстрактный:

Сторонники гибкой разработки программного обеспечения утверждают, что важное ценностное предложение гибких методов заключается в том, что они повышают мотивацию и удовлетворенность сотрудников своей работой. Хотя в ряде исследований представлены отдельные примеры того, что гибкие методы повышают мотивацию и удовлетворенность, исследования не проводили теоретического или эмпирического объяснения связи между методами гибкой разработки и отношением членов команды к своей работе. Опираясь на литературу по менеджменту и разработке программного обеспечения, мы формулируем модель проектирования рабочих мест, которая связывает методы гибкой разработки с восприятием характеристик работы и, таким образом, повышает удовлетворенность работой членов гибкой команды. Используя данные, полученные от 252 специалистов по разработке программного обеспечения, мы протестировали модель и обнаружили положительную связь между методами гибкого управления проектами и разработки программного обеспечения и восприятием сотрудниками характеристик работы. Кроме того, мы обнаружили прямую связь между использованием методов гибкой разработки и удовлетворенностью работой. Наконец, мы обнаружили эффекты взаимодействия между использованием методов гибкого управления проектами и разработки программного обеспечения и восприятием автономности работы. С помощью данного исследования мы вносим вклад в литературу, теоретически объясняя и напрямую оценивая влияние гибких методов разработки на восприятие людьми характеристик своей работы и на их удовлетворенность работой.

Ключевые слова: гибкая разработка, удовлетворенность работой, модель характеристик работы.

Фред Нидерман был старшим редактором, принявшим решение о публикации. Статья была представлена 5 ноября 2014 года и прошла три редакционных редактирования.

1 Введение

За последнее десятилетие всё больше компаний стали применять гибкие методы разработки программного обеспечения (West, Grant, Gerush, & D'Silva, 2010). Независимо от того, отстаивают ли они такие гибкие методы, как Scrum, Extreme Programming (XP), Crystal Clear Feature Driven Development (FDD) или Kanban, сторонники гибких методов делают два основных утверждения. Во-первых, они утверждают, что гибкие методы позволяют создавать более качественное программное обеспечение — утверждение, подтверждённое исследованиями, изучающими успешность проектов, качество программного обеспечения и другие показатели (например, Maruping, Venkatesh, & Agarwal, 2009a; Maruping, Zhang, & Venkatesh, 2009b; Nevo & Chengalur-Smith, 2011). Во-вторых, они утверждают, что использование гибких методов повышает удовлетворенность сотрудников (Highsmith, 2002) — утверждение, которое исследователи редко подвергали эмпирическому исследованию.

Ограниченное число исследований связывает гибкую разработку (т.е. использование гибких методологий и связанных с ними практик) с удовлетворенностью членов команды разработчиков программного обеспечения. Одно исследование разработчиков программного обеспечения показало, что удовлетворенность была выше при парном программировании (гибкой практике разработки), чем при традиционном индивидуальном программировании (Balijepally, Mahapatra, Nerur, & Price, 2009). Эти авторы предполагают, что «более высокая удовлетворенность пар программирования влияет на снижение текучести кадров» (Balijepally et al., 2009, стр. 111). В исследовании команд другая группа авторов изучала взаимосвязь между использованием гибких практик и мотивацией команды (McHugh, Conboy, & Lang, 2011). Они обнаружили, что три гибкие практики (т.е. планирование итераций, ежедневные стендапы и ретроспективы итераций) были связаны с такими характеристиками работы, как самостоятельность, разнообразие навыков и обратная связь. Аналогично, Tessem и Maurer (2007) Было обнаружено, что некоторые члены конкретной гибкой команды имели более высокое восприятие характеристик работы и удовлетворенности ею, а другие — нет. Эти и другие исследования методов гибкой разработки подтверждают их влияние на восприятие работы, включая удовлетворенность, но они имеют существенные теоретические и методологические ограничения (например, Layman, Williams, & Cunningham, 2004; Mannaro, Melis, & Marchesi, 2004; Mann & Maurer, 2005; McHugh, Conboy, & Lang, 2010).

Например, некоторые исследователи основывали свои выводы на результатах одной или нескольких команд (например, McHugh & Lang, 2011; Tessem & Maurer, 2007). Более того, многие исследователи изучали влияние одной или нескольких практик, перечисленных в методологии Agile. Например, McHugh et al. (2012) сосредоточились на трёх практиках метода Scrum (планирование спринта/итерации, ежедневные стендапы и ретроспективы спринта/итерации). при исследовании доверия в гибких командах. Тессем и Маурер (2007) использовали одну гибкую команду, применявшую метод Scrum, для изучения влияния гибких методов на удовлетворенность работой членов команды. Хотя Мельник и Маурер (2006) провели большую выборку (n = 448) для сравнения удовлетворенности работой членов команд, использующих гибкую разработку, и членов традиционных команд, их исследование было описательным, не оценивало использование гибких методов и не проверяло механизм влияния гибких методов на удовлетворенность работой. Следовательно, хотя некоторые данные указывают на связь между гибкими методами и удовлетворенностью, важно провести тщательное, теоретически обоснованное полевое исследование взаимосвязи между гибкими методами разработки и удовлетворенностью работой.

Нам необходимо глубже понять взаимосвязь между методами гибкой разработки и удовлетворенностью работой, поскольку исследования в области информационных систем (ИС), маркетинга и менеджмента неизменно показывают, что дизайн работы связан с удовлетворенностью сотрудников, что приводит к положительным результатам для организаций-работодателей. Метаанализ исследований отношения сотрудников показал, что дизайн работы влияет на удовлетворенность работой, которая, в свою очередь, влияет на производительность труда (Harrison, Newman, & Roth, 2006). В исследованиях рабочей силы в сфере информационных технологий (ИТ) значительный объем исследований связывает дизайн работы с такими результатами, как внутренняя мотивация, удовлетворенность работой и намерение текучести кадров (Joseph, Ng, Koh, & Ang, 2007; Thatcher, Liu, Stepina, Goodman, & Treadway, 2006; Thatcher, Stepina, & Boyle, 2002). Поскольку гибкие методы и связанные с ними практики изменяют структуру работы разработчиков программного обеспечения (Tessem & Maurer, 2007) — например, в парном программировании (Cockburn & Williams, 2001), первоначальной разработке тестов (Beck, 2003) и итеративной поставке (Мое, Dingsoyr, & Dyba, 2010) — мы предполагаем, что использование этих методов влияет на мотивацию и удовлетворенность сотрудников.

В данной статье мы исследуем взаимосвязь между использованием гибких методологий и удовлетворенностью работой членов гибких команд. Хотя сторонники гибких методологий заявляют об общей философии, описанной в манифесте гибких методологий (Fowler & Highsmith, 2001), каждый метод возник в разных обстоятельствах и имеет свою направленность. Например, Scrum (Schwaber & Beedle, 2002) — это, прежде всего, метод управления проектами, в то время как экстремальное программирование (XP) (Beck, 2000) больше фокусируется на вопросах разработки программного обеспечения. В силу разного наследия каждый метод предписывает разные, а иногда и противоречивые практики. Например, в то время как XP предписывает коллективное (командное) владение кодом, разработка на основе функций (FDD) (Palmer & Felsing, 2002) предписывает, чтобы у каждого компонента кода был определенный, единственный владелец. Мы применяем термин «практики» к конкретным гибким методам, таким как парное программирование и коллективное владение кодом.

Напротив, «методы» относятся к любому из ряда определенных взаимозависимых наборов практик, разработанных специалистами по программному обеспечению, таких как XP, FDD и Scrum.

Кроме того, хотя практикующие Agile заявляют о применении методов, лишь немногие применяют все практики одного метода (Conboy & Fitzgerald, 2010), а многие перенимают практики из нескольких методов (VersionOne, 2011). В связи с этим различием в применении, даже среди команд, заявляющих о применении одного и того же метода, в данной работе мы в первую очередь сосредоточимся на использовании конкретных Agile-практик и их влиянии на удовлетворенность работой. При этом мы исследуем два исследовательских вопроса. Во-первых, сторонники Agile утверждают, что Agile-метод влияет на удовлетворенность работой из-за комбинированного и перекрывающего эффекта нескольких практик (Beck, 2000; Highsmith, 2002). Эти практики могут не в равной степени или вообще не способствовать удовлетворенности работой. Поэтому мы рассматриваем следующий исследовательский вопрос:

RQ1: Как степень использования различных гибких практик влияет на удовлетворенность работой?

Для исследования этого вопроса мы разработали теоретическую модель, связывающую гибкие практики с мотивацией и удовлетворенностью работой членов команды, использующей гибкую разработку. Первоначальные исследования применения гибких методов, мотивации и удовлетворенности работой показали, что члены гибких команд более мотивированы (McHugh & Lang, 2011) и удовлетворены своей работой, чем члены команд, использующих традиционную разработку (Melnik & Maurer, 2004). Общая тема этой литературы — то, что гибкие практики влияют на восприятие членами команды характеристик работы, что является ключевым фактором, предсказывающим удовлетворенность работой (Hackman & Oldham, 1980). Мы используем существующий массив знаний о проектировании рабочих мест и управлении ИТ-персоналом (например, Ахунджа, Чудоба, Кацмар, Макнайт и Джордж, 2007; Мур, 2000; Моррис и Венкатеш, 2010; Тэтчер и др., 2002), чтобы обосновать модель, которая связывает использование популярных гибких практик и процессов с удовлетворенностью работой как напрямую, так и косвенно, через их влияние на восприятие.

характеристик работы. Поэтому мы разработали модель, которая исследует наш второй исследовательский вопрос:

RQ2: Каковы причинно-следственные связи, посредством которых гибкие практики влияют на удовлетворенность работой?

В данной работе мы разрабатываем и тестируем модель, которая оценивает влияние гибких методов на удовлетворенность работой и то, как использование гибких методов влияет на восприятие сотрудниками характеристик работы. Эта модель вносит вклад в литературу по гибкой разработке и информационным системам, 1) исследуя предпосылки восприятия работы, 2) признавая, что гибкие методы и их определенные практики в значительной степени являются примерами проектирования работы, и 3) разрабатывая теоретическую призму, через которую можно объяснить влияние гибких методов на отношение к работе. В разделе 2 мы рассмотрим теоретические основы данного исследования.

2 Что такое команда гибкой разработки?

Команды гибкой разработки (ADT) – это группы людей, которые совместно работают над созданием новых и модификацией существующих программных систем. ADT обычно включают в себя участников с широким набором навыков, таких как анализ, программирование, проектирование, архитектура и администрирование баз данных, системная инженерия и управление проектами. Фактически, гибкие команды исходят из того, что все, кто нужен команде для выполнения всех ролей, необходимых для завершения проекта, должны участвовать в проектной команде (Highsmith, 2002). Эта ситуация отличается от традиционных каскадных/структурированных сред, где команды часто специализируются в соответствии с функциями (например, анализ, проектирование, разработка, тестирование) (Nerur, Mahapatra, & Mangalaraj, 2005)¹.

Члены команды ADT часто имеют разные обязанности и должности в разных организациях. Например, системные аналитики могут выполнять обязанности по программированию и проектированию, даже если их должностные обязанности изначально не разграничены. Разработчик же может выполнять задачи программирования, тестирования и управления конфигурацией, которые в традиционной среде распределены фрагментарно. Поэтому мы используем термин ADT для обозначения множества должностей и ролей, выполняемых командой, разрабатывающей программный продукт.

ADT используют гибкие методы, такие как Scrum, XP или Kanban, или комбинации практик из нескольких гибких методов (Highsmith, 2002; VersionOne, 2011). ADT обычно работают короткими циклами, в которых они поставляют работающее программное обеспечение постепенно и итеративно, и фокусируются на понимании проблемной области в процессе разработки (Beck, 2003; Cockburn, 2001; Harris, Collins, & Hevner, 2009; Schwaber & Beedle, 2002). В отличие от традиционных команд, гибкие команды не фокусируются на определении системных требований и проектировании заранее или на полном понимании проблемной области на ранних этапах (Boehm, 1984), поскольку им не хватает более длительных циклов поставки, используемых традиционными методами управления проектами (Boehm & Turner, 2004; Highsmith, 2002).

¹ Мы используем термины «традиционный» и «водопадный» или «структурированный» в качестве сокращенного обозначения традиционных подходов к разработке, но понимаем, что «водопадный» как таковой не обязательно реализуется полностью и не исключает некоторого уровня итерации во многих организациях.

Несмотря на то, что многие гибкие методы требуют активного участия клиента в ADT (например, практика XP по работе с клиентами на месте) – а в некоторых случаях и участия клиента в ADT – мы не включаем клиентов в нашу концепцию ADT по двум причинам. Во-первых, не все ADT включают клиента в состав команды. Хотя все гибкие методы предполагают постоянную доступность владельца продукта для принятия решений, для большинства клиентов ADT нетипично быть полноправными членами команды (Cockburn, 2001; Highsmith, 2002). Во-вторых, заявления сторонников Agile об удовлетворенности работой касаются членов ADT, входящих в основную команду, поставляющую программное обеспечение (Beck, 2000; Highsmith, 2002). Таким образом, разработка нашей теории сосредоточена на ролях ADT, которые обычно являются частью функции информационных технологий в организациях: разработчики программного обеспечения, аналитики по обеспечению качества и тестированию, бизнес-аналитики, архитекторы программного обеспечения, менеджеры проектов и руководители групп.

Подводя итог, можно сказать, что ADT 1) представляет собой кросс-функциональную команду, которая разрабатывает и обновляет программное обеспечение; 2) преимущественно использует гибкие методы управления и разработки, такие как итеративная поставка или парное программирование; и 3) состоит из членов, чьи обязанности сосредоточены на поставке программного обеспечения, а не на внешнем управлении. команда или бизнес.

2.1 Модель характеристик работы и гибкие команды разработки

Используя практики, описанные в гибких методах, организации меняют структуру работы членов ADT. Например, гибкие методы требуют от разработчиков программного обеспечения участия в деятельности, выходящей за рамки программирования, такой как системный анализ и управление проектами, что, вероятно, повысит уровень их воспринимаемой самостоятельности в работе, а также разнообразие навыков и задач. Более того, в литературе по менеджменту и информационным технологиям предполагается, что, изменяя характеристики работы, гибкие методы могут влиять на мотивацию, удовлетворенность работой и производительность членов ADT.

Модель характеристик работы (JCM) (Hackman & Oldham, 1980; Joseph et al., 2007) представляет собой полезный инструмент для понимания того, как изменения, вызванные гибкими методами, влияют на удовлетворенность работой. JCM определяет пять характеристик работы, которые влияют на восприятие работы работниками и их отношение к ней: 1) значимость задачи, которая показывает, в какой степени люди верят, что их работа влияет на жизни людей — как общества в целом, так и организации; 2) идентичность задачи, которая показывает, в какой степени задачи работы являются «целостными» или предполагают достижение определенного результата; 3) разнообразие навыков, которое показывает, в какой степени человек воспринимает работу как требующую разнообразных навыков, талантов и опыта; 4) автономность работы, которая показывает, в какой степени у работника есть право решать, как выполнять требуемую работу, и устанавливать график ее выполнения; и 5) обратная связь, которая показывает, в какой степени процесс выполнения работы предоставляет работникам информацию, с помощью которой они могут оценить свою собственную производительность.

Исследования использовали модель JCM для объяснения удовлетворенности работой ИТ-специалистов (например, Моррис и Венкатеш, 2010), намерения уйти в отставку (например, Тэтчер и др., 2002) и усталости от работы (например, Мур, 2000), а также других показателей. Удовлетворенность работой – это эмоциональная реакция, возникающая в результате трудового опыта (Вайс и Кропанцано, 1996), в то время как усталость от работы – это истощение эмоциональных, умственных и физических ресурсов в результате трудового опыта (Мур, 2000). Исследования показали, что усталость от работы связана с более низкой удовлетворенностью работой (Берк и Грингласс, 1995; Ратнер, Хардгрэйв и Макнайт, 2008) и более высокой текучестью кадров (Мур, 2000). Характеристики работы тесно связаны с удовлетворенностью работой и усталостью от нее (например, Ахуджа и др., 2007; Энг и Слотер, 2001; Мур, 2000; Моррис и Венкатеш, 2010; Ратнер и др., 2008; Тэтчер и др., 2002). Исследования текучести кадров среди широких групп ИТ-специалистов (Дингер, Тэтчер, Тредуэй, Степина и Бреланд, 2015; Мур, 2000; Тэтчер и др., 2006) и более узких групп, таких как мобильная ИТ-сфера (Ахуджа и др., 2007) и разработчики программного обеспечения (Плай, Мур, Уильямс и Тэтчер, 2012),

обнаружили, что JCM полезен для объяснения удовлетворенности работой, но у нас нет строгих исследований того, как гибкие методы влияют на восприятие сотрудниками характеристик работы, или удовлетворенность работой встречается редко (для исключения см. Педрич, Руссо и Суччи, 2011).

Таким образом, чтобы изучить заявления практиков гибкой разработки программного обеспечения о том, что их методы приводят к более высокой удовлетворенности работой, мы используем JCM как линзу, через которую можно теоретически рассуждать о влиянии гибких методов разработки программного обеспечения. о восприятии и удовлетворенности своей работой членами ADT. В таблице 1 представлены определения характеристик работы и других конструктов, используемых в данном исследовании.

Таблица 1. Конструкции, использованные в этом исследовании

Имя конструкции	Определение (источник)
Зависимая переменная	
Удовлетворенность работой	Степень положительной эмоциональной реакции на работу, возникающая в результате оценки работником работы как приносящей удовлетворение или соответствующей его ценностям (Моррис и Венкатеш, 2010; Вайс и Кропанцано, 1996).
Восприятие характеристик работы	
Автономия работы	Степень, в которой работа предоставляет работникам свободу выбора в выборе способа выполнения работы и установлении графика выполнения рабочих задач (Hackman & Oldham, 1980; Moore, 2000; Thatcher et al., 2002).
Обратная связь	Степень, в которой выполнение трудовой деятельности обеспечивает работникам с четкой информацией об их собственных результатах (Хэкман и Олдхэм, 1980; Мур, 2000; Тэтчер и др., 2002).
Разнообразие навыков	Степень, в которой работа требует использования различных талантов (Hackman & Oldham, 1980; Moore, 2000; Thatcher et al., 2002).
Идентификация задачи	Степень, в которой работа предполагает достижение целостного идентифицируемого результата (Хэкмен и Олдхэм, 1980; Мур, 2000; Тэтчер и др., 2002).
Значимость задачи	Степень влияния работы на жизнь людей в организации или обществе в целом (Hackman & Oldham, 1980; Moore, 2000; Thatcher et al., 2002).
Независимые переменные	
Гибкие методы управления проектами	Степень, в которой команда респондента использует гибкие методы управления проектами, определенные в данном исследовании, а именно: выгорание, ежедневные стендапы, ретроспективные совещания и итеративную поставку.
Практика управления гибким подходом к разработке программного обеспечения	Степень, в которой команда респондента использует методы гибкого подхода к разработке программного обеспечения, определенные в данном исследовании, а именно: парное программирование, автоматизированные сборки, непрерывную интеграцию, модульное тестирование, рефакторинг, стандарты кода.

Мы утверждаем, что ADT организуют работу таким образом, что это приводит к более позитивному восприятию их членами Характеристики работы. Технологические модели развития (ТАР) реализуют это, реализуя пять принципов проектирования работы, предложенных Хэкменом и Олдхэмом (1980), которые способствуют более глубокому восприятию характеристик работы и повышению удовлетворенности ею: 1) объединение задач, 2) формирование естественных рабочих единиц, 3) установление отношений с клиентами, 4) вертикальная загрузка работы и 5) открытие каналов обратной связи (рис. 1). Сторонники ТАР утверждают, что принципы научного управления, воплощаемые в традиционных методах разработки программного обеспечения, дробят работу, снижают воспринимаемые характеристики работы и снижают мотивацию и удовлетворенность.

Гибкие методы могут способствовать дефрагментации процессов разработки программного обеспечения, требуя от членов команды объединения навыков и задач, а также разделения ответственности. Это приводит к улучшению восприятия характеристик работы и связанных с этим установок, таких как удовлетворенность работой. Например, вместо подхода «сборочной линии», при котором каждый работник выполняет небольшую часть работы, а затем передает ее следующему, гибкие методы требуют, чтобы каждый член ADT брал на себя ответственность за создание программного обеспечения. Таким образом, гибкие методы помогают членам ADT лучше понимать объем и значимость выполняемой работы (Hackman & Oldham, 1980). В разделе 2.2 мы описываем каждый из принципов проектирования работы и сопоставляем конкретные гибкие методы с принципами проектирования работы (см. рисунок 1), что приведет к улучшению восприятия характеристик работы.

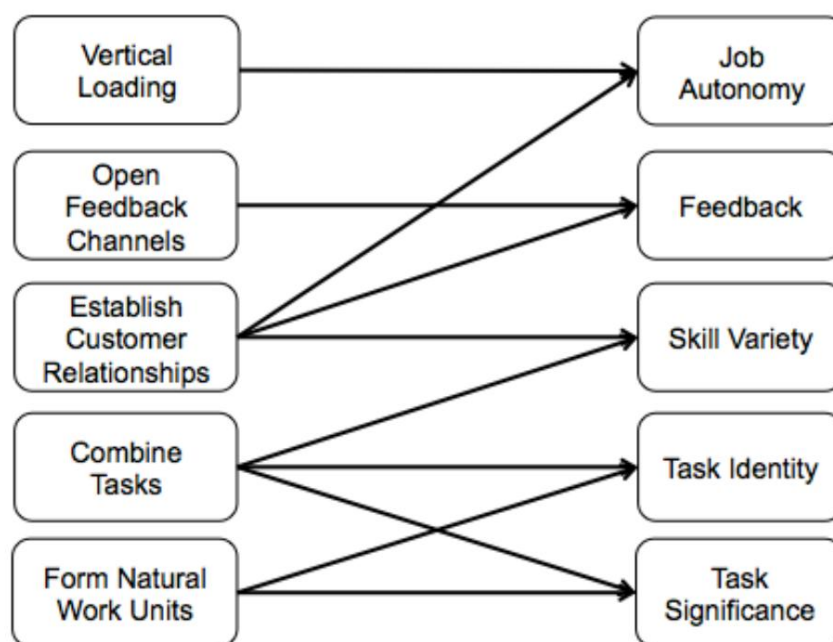


Рисунок 1. Принципы проектирования рабочих мест в связи с характеристиками рабочих мест (Хэкман и Олдхэм, 1980)

2.2 Методы Agile как способ перепроектирования работы

Методы Agile предписывают широкий спектр практик, которые могут влиять на восприятие характеристик работы. Мы фокусируемся на практиках, а не на методах, поскольку команды, работающие по Agile, рассматривают методы как набор инструментов, к которым они могут обращаться по мере необходимости (Conboy & Fitzgerald, 2010). Хотя некоторые команды идентифицируют себя как использующие тот или иной конкретный метод, значительная часть команд использует гибридные методы, включающие практики из нескольких методов (VersionOne, 2011). Хотя разработчики в средах Agile-разработки используют не менее 25 практик, более половины из них используют только 11 (VersionOne, 2011). В связи с их высокой распространенностью, мы включаем эти 11 практик в наш обзор Agile и проектирования рабочих мест.

Кроме того, мы также включаем парное программирование (16-е место) в связи с его влиянием на характеристики работы (например, Tessem & Maurer, 2007). В таблице A3 представлены ранг и процент использования этих 12 и других гибких практик.

Мы предлагаем краткие определения гибких практик, которые мы рассматриваем в данном исследовании (см. Таблицу 2). Практики гибкой разработки можно разделить на две основные категории (Tripp & Armstrong, 2014)². Первая относится к управлению проектами и включает ежедневные стендапы, итеративную поставку, ретроспективы и Burndown-тесты. Вторая относится к подходу ADT к разработке программного обеспечения (SDA) и включает автоматизированное (модульное) тестирование, автоматизированную сборку, непрерывную интеграцию, стандарты кодирования и рефакторинг. Как мы уже отмечали выше, в связи с доказательствами его потенциального влияния на удовлетворенность работой, мы включаем парное программирование в категорию гибких SDA (Таблица 2). Мы предлагаем, чтобы эти гибкие практики реализовывали принципы проектирования задач, как показано в Таблице 3.

Подводим итоги. Мы описываем предполагаемое влияние использования каждой гибкой практики на принципы проектирования рабочих мест, начиная с объединения задач, затем формирования естественных рабочих подразделений, установления отношений с клиентами, вертикальной загрузки задач и открытия каналов обратной связи. Сначала мы описываем каждый принцип проектирования рабочих мест. Во-вторых, мы приводим конкретную иллюстрацию применимых методов управления проектами или разработки программного обеспечения, связанных с этим принципом проектирования работ.

² Трипп и Армстронг провели двухэтапное испытание по категоризации 12 гибких практик, привлекая трёх экспертов по гибкой разработке. Судьи выделили две категории: гибкое управление проектами и гибкий подход к разработке программного обеспечения. Судьи достигли 97,2-процентного согласия в отношении категоризации.

Таблица 2. Определения гибких практик, описанных в данном исследовании

Название Agile-практики	Определение (источник)
Практики управления проектами	
Ежедневное совещание (Швабер и Бидл, 2002)	Ежедневное (обычно) совещание, на котором все участники проекта встречаются стоя, чтобы обеспечить краткость. В Scrum-методологии совещание включает в себя постановку трёх вопросов и ответы на них*. 1. Чего я достиг вчера? 2. Что я буду делать сегодня? 3. Какие препятствия мешают моему прогрессу? Ежедневный стендап Kanban задает вопросы и отвечает: 1. Какие препятствия мешают моему прогрессу? 2. Какой прогресс достигнут? * На встрече основное внимание уделяется этим вопросам настолько, насколько это возможно с практической точки зрения.
Итеративная доставка Планирование релиза Планирование итераций Скорость	Процесс 1) планирования и 2) реализации поэтапно. В частности, концепция, согласно которой поставка кода небольшими порциями позволяет команде генерировать код и сразу же получать обратную связь от среды после каждой итерации. Скорость достигается после того, как команда выполнила несколько итераций, и измеряется объём работы, который она может выполнить за одну итерацию.
Ретроспективы (Швабер и Бидл, 2002)	Встреча, проводимая в конце каждой итерации, на которой команда 1) критически осмысливает последнюю итерацию и 2) выявляет и реализует возможности постоянного совершенствования.
Выгорание	Визуальное представление, предоставляющее команде информацию о 1) выполненной работе и 2) работе, которую ещё предстоит выполнить в текущей итерации или выпуске.
Практики подхода к разработке программного обеспечения	
Автоматизированное (модульное) тестирование (Бек, 2000)	Использование специального тестового кода, который можно запустить (обычно автоматически) для проверки влияния изменений в системе. Команда обычно проводит это тестирование до того, как членам ADT разрешается вносить изменения в код, что позволяет разработчикам чтобы убедиться, что они ничего не сломали в системе.
Автоматизированные сборки (Кюстер, Гшвинд и Циммерманн, 2009 г.)	Использование скрипта кода для пересборки программного обеспечения, что позволяет команде быть уверенной в том, что все файлы, необходимые для успешной сборки программного обеспечения, добавлены в репозиторий кода.
Непрерывная интеграция (Бек, 2000; Дюваль, Матяс и др.) Гловер, 2007)	Процесс систематического и регулярного создания и развертывания кода на тестовом сервере.
Стандарты кодирования (Бек, 2000)	Набор установленных норм относительно кодового наименования и согласованности.
Рефакторинг (Фаулер, 1999)	Обязательство команды использовать методы, ведущие к устранению избыточности, устранению неиспользуемых функций и обновлению устаревших проектов.
Парное программирование (Бек, 2000; Кокберн и Уильямс, 2001)	Практика совместной работы двух разработчиков над созданием части кода.

Таблица 3. Принципы проектирования работ и гибкие методы

Agile-практика	КТ	ФНВУ	ЕКР	ВЛ	ОФК
Практики управления проектами					
Ежедневное совещание			х		х
Итеративная доставка Планирование релиза Планирование итераций Скорость		х	х	х	
Ретроспективы				х	х
Выгорание					х
Подход к разработке программного обеспечения					
Автоматизированное (модульное) тестирование	х				х
Автоматизированные сборки	х				х
Непрерывная интеграция					х
Стандарты кодирования	х				
Рефакторинг	х			х	
Парное программирование	х				х
СТ: объединение задач, ФНВУ: формирование естественных рабочих единиц, ЕКР: установление отношений с клиентами, ВЛ: вертикальная загрузка заданий, ОФК: открытие каналов обратной связи					

2.3 Объединение задач

Для правильной разработки кода команды разработчиков программного обеспечения должны писать код, тестировать его и обеспечивать возможность его развертывания в том виде, в котором он написан и протестирован. Традиционная разработка программного обеспечения использует подход, основанный на дроблении работы. Работа распределяется между командами разработчиков, которые фокусируются на определенных уровнях программного обеспечения (база данных, бизнес-логика, пользовательский интерфейс) или этапах жизненного цикла проекта. Например, такие действия, как кодирование, тестирование и развертывание программного обеспечения, часто сегментируются на различные этапы жизненного цикла проекта и выполняются тремя различными командами. Разделяя работу между командами, направлениями и этапами жизненного цикла, традиционные методы разработки ограничивают возможность разработчиков программного обеспечения видеть результаты задач, связанных с готовым продуктом.

Команды, работающие по принципам Agile, организуют работу совершенно иначе. ADT-группы рекомбинируют обязанности разработчиков программного обеспечения, включая в них выполнение нескольких задач и разработку нескольких уровней программного обеспечения, что способствует лучшему пониманию всей системы, а не только разработки отдельного компонента (Anderson, 2004). Поскольку ADT стремятся обеспечить возможность развертывания кода после каждой итерации, весь этот процесс происходит непрерывно. Мы предполагаем, что четыре из выбранных гибких практик связаны с восприятием рекомбинации задач и более высоким восприятием характеристик работы (таблица 2, столбец 2).

2.3.1 Автоматизированное (модульное) тестирование и комбинирование задач

Хотя разработчики в традиционных средах разработки программного обеспечения обычно тестируют ПО перед его развертыванием, большинство традиционных команд не пишут постоянные общие наборы кода для тестирования программных систем. Вместо этого каждый разработчик может выполнять начальное модульное тестирование вручную или с помощью специального кода, написанного для тестирования его конкретных изменений. Кроме того, в традиционных командах специализированная команда системных тестировщиков обычно проводит полное тестирование систем позднее.

При использовании фреймворков автоматизированного тестирования АД пишут как модульные, так и системные тесты, которые сохраняют, чтобы все участники АД могли их использовать. Эти тесты объединяют задачи двумя способами. Во-первых, код является примером документации на практике (Бек, 2000; Кюстер и др., 2009), поскольку сами тесты дают представление о функциональности системы. Во-вторых, эти тесты можно запустить в любое время, что позволяет разработчику проверить последствия изменения кода для всей системы. Таким образом, модульные тесты объединяют задачи, ранее выполнявшиеся бизнес-аналитиком и тестировщиком.

2.3.2 Автоматизированные сборки и комбинирование задач

Автоматизированные сборки — это процесс создания скриптов для создания полной и готовой к развертыванию сборки, и многие ADT требуют, чтобы члены команды запускали автоматизированные сборки перед регистрацией изменений в коде (Küster et al., 2009). Когда ADT делают это, разработчик берёт на себя задачи, выходящие за рамки традиционных обязанностей, и выполняет функции менеджера по конфигурации. Автоматизированные сборки помогают гарантировать, что разработчик включил и настроил все зависимости, необходимые для развертывания программного обеспечения.

Этот процесс вертикально загружает ADT, предоставляя контроль над выполнением скриптов, что снижает необходимость в выделенной роли менеджера конфигурации и возлагает ответственность за обеспечение полноты и готовности пакетов к развертыванию на каждого участника ADT. Более того, ADT регулярно запускают автоматизированные процессы сборки, чаще всего до отправки кода разработчиком (Küster et al., 2009). В ходе этого процесса разработчик получает уведомление о том, не нарушили ли его новые изменения какие-либо другие части кодовой базы, и можно ли корректно собрать код на его машине. Таким образом, автоматизированные сборки предоставляют обратную связь непосредственно разработчикам по мере завершения ими кода.

2.3.3 Стандарты кодирования, рефакторинг и комбинирование задач

Термин «стандарты кодирования» относится к обеспечению того, чтобы ADT следовал согласованным рекомендациям при кодировании. Таким образом, разработчики ADT объединяют задачи, стремясь не просто писать работающее программное обеспечение, а программное обеспечение, которое легче рефакторить и которое легче поручить другим (Auer & Miller, 2002). В то время как программный код форматируется в соответствии с предпочтениями множества пользователей, стандарты кодирования требуют от разработчиков гарантировать, что сам код является не личным объектом, а артефактом сообщества (Beck, 2000). Таким образом, задача программирования в ADT включает в себя задачу форматирования кода сообществом.

Рефакторинг относится к практикам, которые устраняют избыточность, исключают неиспользуемую функциональность и обновляют устаревшие проекты (Fowler, 1999). Рефакторинг как гибкий метод отражает стремление команды улучшать структуру и снижать сложность кода по мере необходимости (Beck, 2000). Вся группа разработчиков ADT (а не какая-то конкретная подгруппа, например, архитекторы) выполняет рефакторинг по мере необходимости. Таким образом, рефакторинг объединяет задачи разработки программного обеспечения, которые традиционные методы могут фрагментировать.

2.3.4 Парное программирование и комбинирование задач

Парное программирование — это практика совместной работы двух разработчиков над кодом (Cockburn & Williams, 2001). При парном программировании два разработчика сначала проектируют программное обеспечение, а затем разрабатывают его (Beck, 2000). В некоторых случаях написание тестов представляет собой упрощенную реализацию спецификации проекта (Highsmith, 2002). В таких случаях некоторые рекомендуют, чтобы первый программист писал тесты для кода, а второй предоставлял обратную связь (Beck, 2000). Затем, при программировании реального системного кода, два разработчика меняются ролями. Добавление разработки дизайна к обязанностям программистов объединяет задачи, которые в традиционных структурах команд могли бы быть распределены между аналитиками, архитекторами и разработчиками.

2.4 Формирование естественных рабочих единиц

2.4.1 Значимость и идентичность задач в единицах естественной работы

Фракционализация работы означает, что человек может выполнить только часть работы, необходимой для выполнения полной задачи, что может привести к тому, что работники будут выполнять, казалось бы, не связанные и неестественные наборы задач в течение заданного периода (Hackman & Oldham, 1980). Непонимание взаимосвязи рабочих единиц приводит к снижению идентичности и значимости задач. Напротив, формируя естественные рабочие единицы, сотрудники развивают чувство ответственности за свою работу (Hackman & Oldham, 1980). Развитие такого чувства ответственности может повысить воспринимаемую значимость и ценность работы. Таким образом, формирование естественных рабочих единиц повышает значимость и значимость задач.

В традиционных средах разработки программного обеспечения дробление работы между различными ролями приводит к назначению задач по типу, например, кодирование базы данных, кодирование пользовательского интерфейса или тестирование. В ADT используется другой подход: все задачи, необходимые для выполнения конкретной функции, распределяются в виде пакета. Вместо того, чтобы просто назначить программисту следующую задачу в его специализации (Андерсон, 2004; Палмер и Фельсинг, 2002). Эта практика согласуется с призывом, изложенным в Agile-манифесте, определить ключевым показателем прогресса в проекте разработки как готовый, функциональный код (Фаулер и Хайсмит, 2001). Вместо того, чтобы определять свою работу как «кодирование уровня доступа к данным», программисты в этом примере могут определить свою работу как завершение «функции XYZ», которая является идентифицируемым и функциональным компонентом готовой системы —

достижение, имеющее определённую значимость. Мы предполагаем, что практики, позволяющие программистам итеративно создавать код, с большей вероятностью, чем традиционные практики, приведут к формированию у программистов естественных рабочих единиц и, в конечном счёте, к более чёткому пониманию сущности и значимости задачи.

2.4.2 Итеративная поставка и формирование естественных рабочих единиц

Хотя в исследованиях планирование релиза, итеративное планирование и скорость часто рассматривались отдельно, все они касаются итеративной поставки (см. Таблицу 2). Планирование релиза определяет на высоком уровне порядок развертывания функций. С этой целью ADT и заказчик итеративно планируют каждый рабочий цикл; то есть ADT и заказчик совместно определяют функции, включаемые в следующий цикл, делят их на задачи и оценивают объем работы, который необходимо выполнить. Эти два подхода также учитывают скорость. Скорость

(как гибкая практика) относится к расчету объема работы, которую ADT может выполнить в течение стандартного времени

Рабочий цикл. Таким образом, рассчитанная скорость ADT помогает правильно определить объем работы, выделенный для итерации.

Эксперты по гибким методологиям, участвовавшие в данном исследовании (подробности см. в Приложении А), неоднократно описывали эти три концепции (планирование релиза, планирование итерации и скорость) как взаимозаменяемые, поэтому мы рассматриваем их как единый метод, называемый итеративной поставкой.

Итеративная поставка помогает ADT формировать естественные рабочие единицы. Когда клиент определяет приоритеты, связанные с необходимой функциональностью, команда определяет объем работ, необходимых для полного завершения функции. Вместо того, чтобы концентрироваться на разработке отдельных модулей кода, команда фокусируется на разработке функций, которые, по определению, являются естественными рабочими единицами.

2.5 Установление отношений с клиентами

В традиционном контексте разработки программного обеспечения работа фрагментирована и специализирована, и существует потенциальное разделение ролей между аналитиком, архитектором, программистом и тестировщиком, что часто приводит к образованию нескольких уровней иерархии между сотрудником и заказчиком. В этом случае сотрудники могут не иметь контакта с теми, для кого была выполнена работа. Отсутствие прямого контакта с заказчиком приводит к тому, что обратная связь фильтруется другими, и сотрудники теряют потенциальный источник информации, позволяющий им оценить влияние своей работы, что снижает значимость задач.

В противоположность этому, гибкие практики подчёркивают важность прямого и регулярного (во многих случаях ежедневного) контакта между клиентом и всеми членами команды. Такой контакт способствует установлению отношений с клиентом. Прямая обратная связь от клиента, полученная в ходе таких отношений, вероятно, предоставит наиболее чёткую информацию, помогающую сотрудникам оценить общую успешность своей работы.

Более того, когда сотрудники устанавливают отношения с клиентами, они получают больше разнообразных навыков и большую независимость в работе. В любой рабочей структуре технические или операционные навыки сотрудников могут развиваться благодаря их работе и обучению; однако, когда сотрудники общаются с клиентами, они могут расширять свой спектр навыков, развивая и используя навыки межличностного общения. Кроме того, когда сотрудники получают возможность напрямую управлять личными взаимоотношениями с клиентами, их восприятие независимости в работе улучшается. Развивать такие отношения с клиентами помогают две гибкие практики управления проектами: ежедневные стендапы и итеративная поставка.

2.5.1 Ежедневные совещания и установление отношений с клиентами

Ежедневное совещание-стендап (обычно) – это ежедневное совещание всей группы ADT, которое проводится вставая, чтобы способствовать краткости обсуждений. В зависимости от используемого гибкого метода, ежедневное совещание-стендап фокусируется на кратком списке вопросов, на которые должен ответить каждый член команды (см. Таблицу 3). Группа ADT в целом выполняет эту практику каждый день. Каждый член ADT предоставляет информацию о проделанной работе за предыдущий день, о планах на день и о любых блокирующих или координирующих проблемах (Schwaber & Beedle, 2002). Гибкие методы также предписывают, что по возможности владелец бизнеса должен присутствовать на этом ежедневном совещании.

Предоставляя возможность обсудить статус проекта, ежедневные стендапы позволяют каждому члену команды ADT напрямую взаимодействовать с заказчиком. Каждый член команды может задавать вопросы заказчику, что открывает возможности для ежедневного итеративного диалога и способствует установлению прямых отношений между разработчиком кода и заказчиком. Кроме того, поскольку ежедневные стендапы фокусируются на проблемах (см. Таблицу 3), члены ADT могут быть открытвенны с заказчиком (владельцем бизнеса), что способствует укреплению доверия, важного компонента хороших отношений (Deutsch, 1960; Lewicki & Bunker, 1995; Lewis & Weigert, 1985).

2.5.2 Итеративная доставка и установление отношений с клиентами

Итеративная поставка устанавливает отношения с клиентом, поскольку создаёт среду, в которой заказчик и ADT совместно управляют процессом планирования. Представители клиента и участники ADT совместно участвуют в планировании релиза и итерации. Согласно Scrum и XP, клиент устанавливает приоритеты задач, а ADT оценивает объём работ и составляет график их выполнения.

Кроме того, ADT использует заданную им скорость (Beck, 2000) для определения объёма работы, который он может выполнить за один рабочий цикл. Используя эту метрику, определяемую на основе объёма работы, выполненной в предыдущих циклах (см. Таблицу 3), ADT устанавливает достижимую цель. Выполняя итерации и релизы в обещанные сроки, ADT в целом устанавливает доверительные отношения с клиентом, и каждый член ADT получает выгоду от этих отношений при регулярном взаимодействии с клиентом.

2.6 Вертикальная нагрузка

Во многих традиционных контекстах разработки ответственность за разработку программного обеспечения разделяется между планированием (анализ и проектирование), выполнением (разработка и тестирование) и контролем (управление проектом), что создает разрыв между каждым компонентом работы (Nerur et al., 2005). Вертикальная загрузка заданий сокращает расстояние между выполнением, планированием и контролем данного фрагмента работы, поскольку вертикальная загрузка добавляет «ответственность и контроль, которые ранее были зарезервированы для более высоких уровней управления» к отдельным заданиям (Hackman & Oldham, 1980, стр. 64). Вертикальная загрузка усиливает восприятие автономности работы и личной ответственности, поскольку она возвращает владельцу задания свободу действий для принятия решений относительно планирования работы, управления временем и, в меньшей степени, устранения неполадок, управления кризисами и финансового контроля (Corderly, Morrison, Wright, & Wall, 2010; Hackman, Brousseau, & Weiss, 1976).

В традиционной среде разработки программного обеспечения одна группа сотрудников (например, архитекторы, аналитики) может принимать решения о проектировании и внедрении системы, в то время как другая группа сотрудников (например, разработчики, тестировщики) может завершать выполнение (или внедрение), что указывает на отсутствие вертикальной нагрузки. В отличие от этого, практики ADT, как правило, предполагают вертикальную нагрузку, объединяя проектирование, разработку, тестирование и другие задачи в обязанности одного члена команды. Мы предполагаем, что большое количество гибких практик увеличивает вертикальную нагрузку, что описано в разделах 2.6.1–2.6.3.

2.6.1 Итеративная доставка и вертикальная загрузка

Для планирования итеративной поставки группа ADT должна оценить объём работы, которую необходимо выполнить. Когда члены ADT участвуют в планировании и оценке, их роли распределены по вертикали. ADT трансформирует разработку программного обеспечения, передавая контроль над планированием и оценкой от руководства или других лиц членам ADT. Передавая управление, ADT вводит больше вертикального контроля в работу членов команды.

2.6.2 Ретроспективы и вертикальная загрузка

Ретроспективные встречи проводятся в конце рабочего цикла. На этих встречах члены ADT анализируют рабочий процесс, использованный в предыдущей итерации, и предлагают или принимают изменения к нему для следующего рабочего цикла. Этот процесс позволяет ADT самостоятельно определять свою деятельность и, таким образом, возлагает на ADT ответственность за определение собственного процесса разработки (McHugh & Lang, 2011; Schwaber & Beedle, 2002). Этот процесс также предоставляет каждому члену право голоса при оценке качества работы всей ADT (Beck, 2000; Highsmith, 2002; McHugh & Lang, 2011; Schwaber & Beedle, 2002).

2.6.3 Рефакторинг и вертикальная загрузка

Рефакторинг подразумевает внесение изменений во внутреннюю структуру программной системы, чтобы сделать системный код более понятным и менее затратным на поддержку и модификацию (Fowler, 1999). Вместо того, чтобы полагаться на руководителя разработки, например, руководителя группы или архитектора, для выполнения этой работы, гибкая методология разработки программного обеспечения (SDA) при рефакторинге основана на вертикальной загрузке этой важной практики. Каждый разработчик в ADT; рефакторинг в ADT — это коллективная деятельность. Вместо того, чтобы предлагать архитектору или другому руководителю изменить существующий код, разработчик ADT может выполнить рефакторинг существующего кода. Гибкие методологии поощряют использование этой практики для мотивации членов команды. Чувствовать, что у них есть рабочая автономия, позволяющая гарантировать, что в программной системе будет как можно меньше дефектов и архитектурных проблем.

2.7 Открытие каналов обратной связи

Сотрудникам необходимо получать дополнительную информацию, касающуюся планирования и выполнения работы, а также её результатов. Поскольку обратная связь — одна из характеристик работы, способствующих повышению удовлетворённости (см. Таблицу 1), дополнительные возможности и способы получения обратной связи должны усилить это восприятие.

Личные отношения и общение обеспечивают обратную связь (Hackman & Oldham, 1980); кроме того, гибкие методы позволяют интегрировать обратную связь в саму работу. Например, когда работа сотрудника по производству продукта или предоставлению услуги включает в себя контроль качества, он может получить прямую обратную связь и дополнительную информацию о своей работе. Методы гибкой разработки, в частности, объединяют проектирование, разработку и тестирование в единый процесс, который обычно выполняется одним и тем же программистом или парой.

Эти процессы разработки, наряду с поддержкой автоматизированных инструментов, позволяют немедленно проверить качество набора кода и предоставить обратную связь непосредственно тому, кто его создал.

2.7.1 Ежедневные стендап-встречи и открытие каналов обратной связи

Ежедневное совещание открывает множество каналов обратной связи, позволяя членам ADT немедленно

Оцените качество своих предыдущих и текущих рабочих процессов и результатов. Когда члены ADT ежедневно обсуждают свой статус со всеми членами команды, они могут обмениваться отзывами о выполненной работе, препятствиях и проблемах, возникших в результате ранее выполненной работы. Эти повторяющиеся, запланированные обсуждения позволяют команде предоставлять и получать обратную связь, а также понимать состояние среды проекта (Burke, Stagl, Salas, Pierce, & Kendall, 2006). Более того, поскольку на встрече присутствует и заказчик, каждый член ADT может получить прямую обратную связь по вопросам, касающимся направления, требований или проблем.

2.7.2 Ретроспективы и открытие каналов обратной связи

Ретроспективные встречи проводятся в конце рабочего цикла и обеспечивают канал обратной связи для членов ADT для оценки качества их работы (Beck, 2000; Highsmith, 2002; McHugh & Lang, 2011; Schwaber & Beedle, 2002). Ретроспективные встречи позволяют ADT постоянно совершенствоваться, поскольку они предоставляют площадку, на которой каждый член команды может описать, что работает хорошо, что не работает хорошо, и как что-то нужно изменить в следующем рабочем цикле (Derby, Larsen, & Schwaber, 2006). Ретроспективы позволяют членам команды прояснить проблемы, возникшие в ходе предыдущих итераций, поскольку эти встречи дают четкую обратную связь о том, достигла ли команда целей предыдущих рабочих циклов и как именно, а также позволяют членам команды определить возможности для улучшения рабочих практик в следующем рабочем цикле (McHugh et al., 2012). Исследования показали, что исключение ретроспективы из процессов ADT приводит к отсутствию единого понимания в команде относительно того, насколько хорошо она работает (Moe et al., 2010).

2.7.3 Выгорание и открытие каналов обратной связи. Диаграмма выгорания — это

канал обратной связи, который сравнивает объём работ, запланированных на определённый момент рабочего цикла, с объёмом фактически выполненных работ (Sutherland, 2001). Вся группа ADT (а не только менеджер проекта) видит эту диаграмму, предоставляя всем членам команды необходимую информацию о

прогресса, достигнутого в достижении ранее поставленной командной цели. Предоставляя ADT обратную связь о текущем прогрессе в достижении цели, диаграмма выгорания позволяет каждому члену ADT принять меры для снижения вероятности отставания проекта.

2.7.4 Автоматизированное (модульное) тестирование, автоматизированные сборки и открытие каналов обратной связи

Эти три подхода работают вместе, открывая каналы обратной связи для каждого разработчика и для всей команды. Во-первых, автоматизированное модульное тестирование открывает каналы обратной связи, предоставляя разработчику мгновенную информацию о неудавшемся тесте. Многие команды разработчиков требуют, чтобы программисты выполнили полный набор тестов перед внесением изменений в общий репозиторий кода (Beck, 2003; Highsmith, 2002; Küster et al., 2009).

Последовательно выполняя все тесты до тех пор, пока они не пройдут успешно, члены ADT получают информацию, которая позволяет им немедленно проверить качество написанного ими кода.

Автоматизированные сборки расширяют возможности автоматизированного (модульного) тестирования. Автоматизированные сборки позволяют любому разработчику пересобрать всю программную систему после обновления из репозитория или в любое другое время перед коммитом. Автоматизированная сборка выходит за рамки автоматизированного модульного тестирования, поскольку практика выполнения автоматизированных сборок гарантирует не только функциональность системного кода, но и то, что команда добавила все остальные зависимости сборки в список системных зависимостей (например, компонент стороннего кода) и в

Скрипты сборки. Этот процесс обеспечивает канал обратной связи для каждого, кто вносит изменения в систему, чтобы команда могла без проблем внести свои изменения в обновлённую систему.

2.7.5 Непрерывная интеграция и открытие каналов обратной связи

Непрерывная интеграция (CI) расширяет каналы обратной связи, предоставляемые отдельным участникам посредством автоматизированного тестирования и автоматизированных сборок, на всю ADT. Поскольку непрерывная интеграция происходит на сервере, общем для всей команды и обновляемом после того, как каждый участник команды вносит код, она позволяет ADT в целом гарантировать единообразие развертывания кода. Это открывает канал обратной связи, позволяющий всему ADT определять, выполнили ли участники изменения должным образом. CI обеспечивает обратную связь, выходящую за рамки автоматизированного тестирования и автоматизированных сборок, поскольку гарантирует не только запуск кода на компьютере разработчика, но и внесение разработчиком всех необходимых изменений конфигурации в репозиторий кода (Duvall et al., 2007; Fowler, 2006).

2.7.6 Парное программирование и открытие каналов обратной связи

Работа в парах позволяет программистам эффективнее выявлять ошибки в процессе кодирования. Когда наблюдатель замечает ошибку в процессе, он может немедленно дать обратную связь для её исправления.

Кроме того, поскольку гибкие методы утверждают, что пары должны быть гибкими структурами, они часто регулярно рекомбинируют (в некоторых случаях пары могут меняться более одного раза в день) (Бек, 2000); таким образом, каналы обратной связи открываются между парой, чтобы гарантировать, что весь ADT последовательно реализует стандарты кодирования.

Подводя итог, в этом разделе дается определение каждого из пяти принципов проектирования рабочих мест и объясняется, как каждая из десяти гибких практик соотносится с каждым из этих принципов. Мы предполагаем, что восприятие характеристик рабочих мест усиливается, когда практики ADT, реализующие принципы проектирования рабочих мест, проектируют и рекомбинируют рабочие места. В разделе 2.8 мы сопоставляем гибкие практики с принципами проектирования рабочих мест и разрабатываем модель, связывающую гибкие практики, характеристики рабочих мест и удовлетворенность работой.

2.8 Внедрение и сопоставление гибких практик с принципами проектирования рабочих мест и моделью характеристик рабочих мест

Таблица 2 классифицирует 10 гибких практик на две группы: практики agile-управления проектами (PM) и практики agile-подхода к разработке программного обеспечения (SDA). Мы считаем, что лучше всего концептуализировать эти группы как отдельные многомерные конструкции второго порядка. Они различны, поскольку agile-практики PM и agile-практики SDA имеют разные фокусы и по-разному влияют на восприятие характеристик работы. Agile-практики SDA сосредоточены в первую очередь на выполнении процесса разработки программного обеспечения. В противоположность этому, agile-практики PM в первую очередь сосредоточены на установлении отношений с клиентами, получении обратной связи и координации работы в командах. Таким образом, agile-практики PM являются в первую очередь управленческими и поддерживают производительность фактической разработки программного обеспечения. Из-за этих разных фокусов можно представить agile-практики как две концептуально связанные, но разные конструкции.

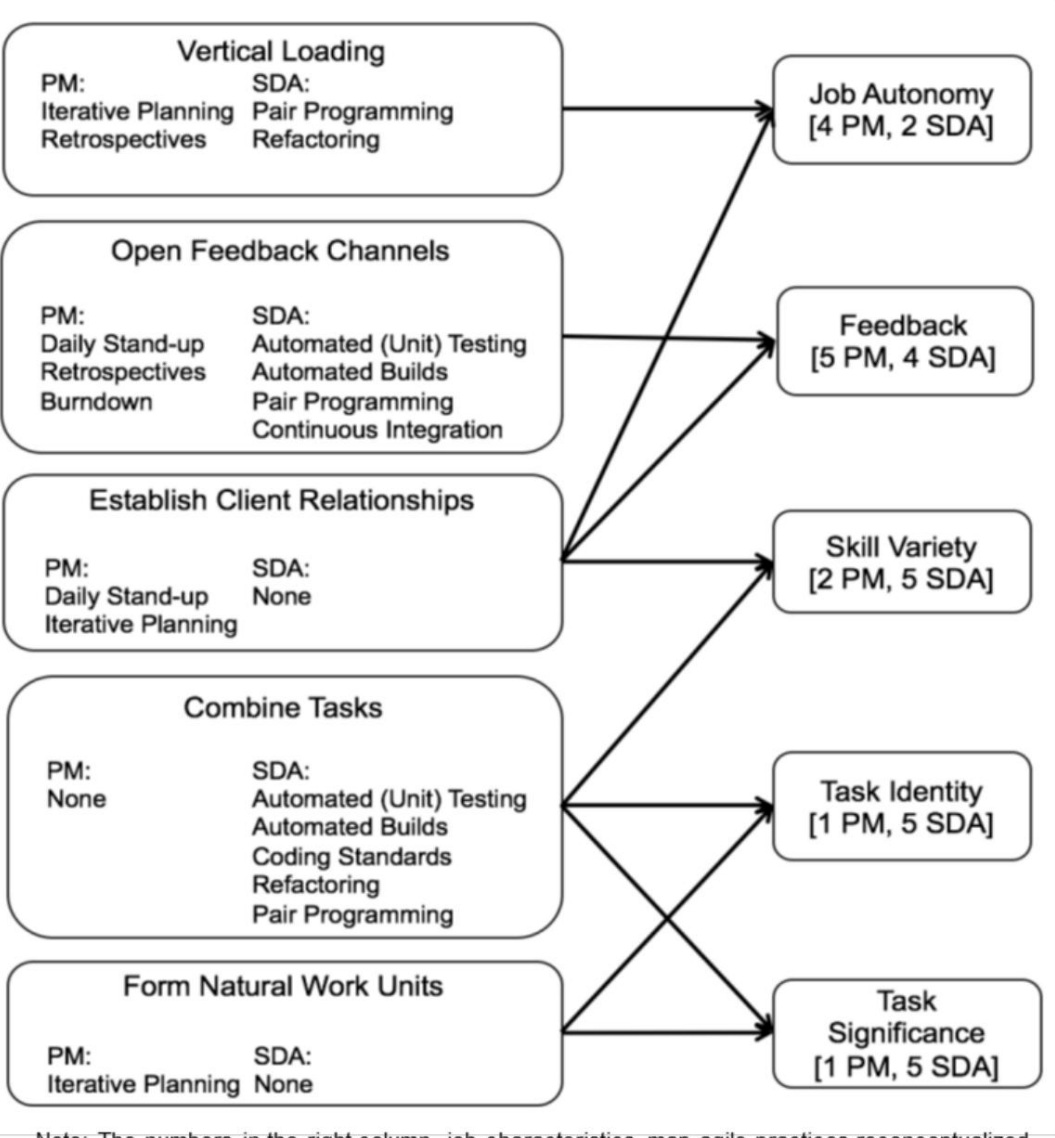
Мы считаем, что необходимо моделировать каждую из 10 отдельных agile-практик как рефлексивную по своей природе (Petter, Straub, & Rai, 2007; Polites, Roberts, & Thatcher, 2012). Поскольку терминология непоследовательна в разных методах и поскольку одна команда может заявлять об использовании практики, но фактически не внедрять ее так же, как другая команда, сложно оценить, насколько часто команды используют каждую практику. С этой целью в нашем опросе (описанном в Разделе 4) мы задали несколько вопросов, специфичных для agile-практик; например, использование стендап-митингов и итеративной поставки помогает определить глубину, в которой ADT использует agile-практики управления проектами. Для этих факторов первого порядка мы задали вопросы, которые охватывали область деятельности команды, чтобы оценить глубину использования каждого гибкого метода. Таким образом, чтобы оценить степень использования стендап-митингов, например, мы задавали такие вопросы, как частота и спектр вопросов, обсуждаемых на встречах. Хотя эти вопросы затрагивают более широкую сферу стендап-встреч, их можно рассматривать как взаимозаменяемые.

Мы концептуализировали факторы второго порядка как формирующие по своей природе (Polites et al., 2012), поскольку использование одной гибкой практики (например, парного программирования) может не требовать одновременного использования других гибких практик.

Ввиду независимости большинства agile-практик, использование одной практики на высоком уровне не обязательно означает, что команда использует другую практику на высоком уровне. Наша концептуализация agile-практик предполагает, что для эффективной оценки того, используют ли команды agile PM и agile SDA, необходимо оценить не только глубину применения практики, но и то, использует ли команда другие agile-практики. Следовательно, если на первом этапе мы использовали несколько взаимозаменяемых элементов для оценки использования практики (например, рефлексивным способом), то на втором этапе концептуализация потребовала также агрегировать наличие конкретных практик, используемых ADT.

По этой причине на втором этапе следует операционализировать гибкие методы управления проектами (PM) и разработки программных продуктов (SDA) в качестве формирующих.

Как предполагает наш обзор и связанная с ним теория измерений, можно концептуализировать практики PM и SDA как агрегированные конструкции; то есть их можно концептуализировать как имеющие рефлексивные измерения первого порядка и формирующие отношения от первого порядка к связанному конструкту второго порядка. На рисунке 2 мы концептуализируем agile-практики как действующие как примеры принципов проектирования рабочих мест. Наше объяснение рисунка дополнительно подчеркивает важность практик моделирования как отдельных конструкций на уровне второго порядка. Agile-практики PM и agile-практики SDA во многом пересекаются в концепциях вертикальной загрузки и открытия каналов обратной связи: более 50 процентов практик каждой категории реализуют эти два принципа проектирования рабочих мест. Напротив, только agile-практики PM реализуют установление отношений с клиентами и формирование естественных рабочих единиц, в то время как только agile-практики SDA реализуют объединение задач.



Note: The numbers in the right column, job characteristics, map agile practices reconceptualized as instances of the job design principles, using Hackman & Oldham's (1980) job characteristics model (JCM). For example, because two job design principles constructs (left column) lead to job autonomy, the instances from both vertical loading and establishing client relationships are added to generate the numbers in the right column. Because some practices are instantiated in multiple ways, the impact of the same agile practice can be counted multiple times.

Рисунок 2. Гибкие практики как примеры принципов проектирования работ, сопоставленных с JCM

Продолжая сопоставление, в правом столбце рисунка 2 перечислены пять характеристик работы и кумулятивные примеры реализации принципов проектирования работы с использованием каждой гибкой практики. Например, в категории «автономия работы» представлены четыре гибкие практики управления проектами (две из области вертикальной загрузки и две из области установления отношений с клиентами).

и две гибкие практики SDA (две из вертикальной загрузки и ни одной из установления отношений с клиентами).

Как показано на рисунке, категории «автономия работы» и «обратная связь» имеют относительно большое количество сопоставленных им практик Agile PM и Agile SDA, в то время как остальные характеристики работы в первую очередь сопоставлены с практиками Agile SDA. Хэкман и Олдхэм (1980) утверждают, что три конструкта – идентичность задачи, разнообразие навыков и значимость задачи – в первую очередь связаны с рабочими задачами, тогда как конструкты обратной связи и автономии работы – с управлением задачами. Аналогичным образом, концепция Agile SDA включает практики, которые в первую очередь ориентированы на сам процесс разработки программного обеспечения, в то время как практики Agile PM сосредоточены на управлении задачами (Tripp & Armstrong, 2014). Последствия этих различий в сопоставлении напрямую влияют на нашу исследовательскую модель, и мы обсудим их в Разделе 3.

Подводя итог, в этом разделе мы описываем модель характеристик рабочих мест Хэкмана и Олдхэма (1980) и принципы проектирования рабочих мест. Мы также иллюстрируем многочисленные способы, которыми практики, определяемые гибкими методологиями, могут рассматриваться как примеры реализации принципов проектирования рабочих мест. Мы концептуализируем практики управления проектами (PM) и управления процессами (SDA) как агрегированные конструкции второго порядка. Наконец, мы представляем сопоставление гибких практик с принципами проектирования рабочих мест.

а затем к модели «характеристики работы». В разделе 3 мы представляем нашу исследовательскую модель и гипотезы.

3. Разработка гипотез

На рисунке 3 представлена наша исследовательская модель, а в таблице 1 (см. раздел 2.1) определены наши конструкции.

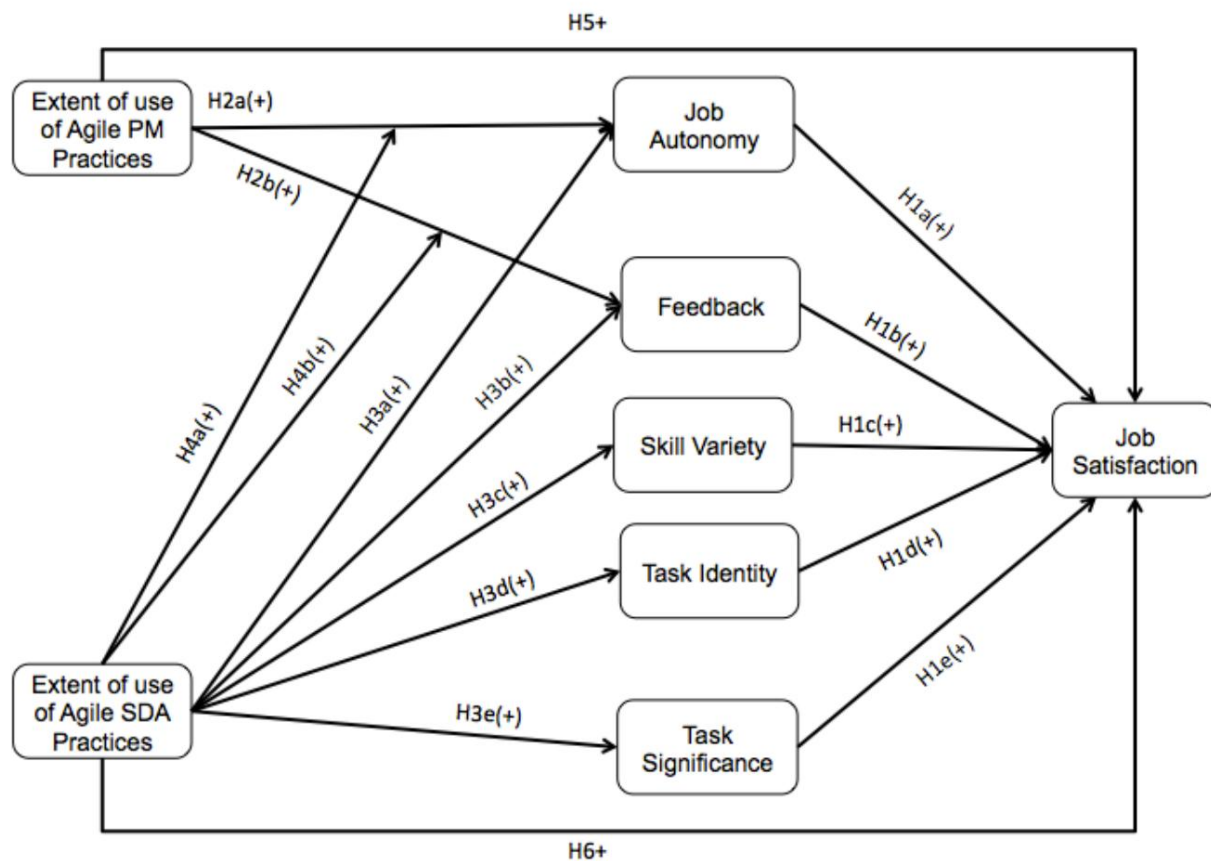


Рисунок 3. Модель исследования

Наша исследовательская модель использует логику, изложенную в предыдущих разделах данной статьи, а также существующие исследования. Исследования тщательно проверили наш первый набор гипотез (например, Ahuja et al., 2007; Morris & Venkatesh, 2010; Thatcher et al., 2002). Мы включили эти гипотезы в качестве средства для связи нашей модели с существующей номологической сетью ИТ-персонала (Joseph et al., 2007). Следовательно, мы предполагаем, что более благоприятное восприятие сотрудниками характеристик работы приводит к более благоприятному восприятию удовлетворенности ею.

В связи с этим мы выдвигаем гипотезу:

H1a-e: Более высокие оценки характеристик работы: а) обратная связь, б) разнообразие навыков, в) идентичность задачи, г) значимость задачи и д) автономность в работе положительно связаны с удовлетворенностью работой.

Наш обзор методов гибкой разработки и принципов проектирования рабочих мест показывает, что при увеличении использования гибких практик члены ADT должны воспринимать свои рабочие места как менее фрагментированные, более автономные и более значимые.

Мы утверждаем, что гибкие методы управления проектами (PM) и планирования задач (SDA) по-разному связаны с конструктами характеристик работы.

Во-первых, мы предполагаем, что степень использования гибких методов управления проектами (PM) в первую очередь влияет на характеристики работы, связанные с управлением работой, а не с выполнением задач по разработке программного обеспечения. Как утверждают Хэкмен и Олдхэм (1980), обратная связь и автономия работы в первую очередь связаны с управлением работой.

Применение гибких практик управления проектами увеличивает вертикальную нагрузку, открывает каналы обратной связи и позволяет всей команде взаимодействовать с клиентом. Эти факторы в первую очередь отражаются в обратной связи и самостоятельности в работе — двух важных характеристиках работы.

Следовательно, применение этих принципов проектирования работы должно повысить позитивное восприятие обратной связи и самостоятельности в работе. В связи с этим мы выдвигаем гипотезу:

H2a-b: Степень использования гибких методов управления проектами положительно влияет на воспринимаемые характеристики работы: а) автономии в работе и б) обратную связь.

В отличие от гибких практик управления проектами (PM), которые фокусируются на управлении проектом, гибкие практики SDA фокусируются на выполнении конкретной задачи. Таким образом, гибкие практики SDA, вероятно, повлияют на характеристики работы, связанные с задачами: значимость задачи, её идентичность и разнообразие навыков. Это происходит, главным образом, за счёт гибких практик, объединяющих задачи, фрагментированные в традиционных моделях работы (например, задачи разработчиков объединяются, когда разработчику разрешается писать тесты и компоненты производственного кода для различных функций, а не передавать тестирование другому человеку).

Кроме того, в соответствии с картой на рисунке 2, мы утверждаем, что гибкие практики SDA также могут влиять на автономность работы и обратную связь. Гибкие практики SDA открывают каналы обратной связи несколькими способами. Во-первых, автоматизированное тестирование и непрерывная интеграция позволяют членам ADT быстро оценивать влияние любого изменения кода. По своей природе парное программирование обеспечивает обратную связь для программиста, поскольку партнёр непрерывно оценивает работу по мере её выполнения. Наконец, гибкие практики SDA предоставляют множество возможностей для вертикальной загрузки задач, что может привести к большему количеству возможностей для достижения автономности.

Поскольку гибкие методы SDA можно сопоставить со всеми пятью характеристиками работы, мы выдвигаем гипотезу:

H3a-e: Степень использования гибких методов SDA положительно влияет на восприятие характеристик работы: а) автономии в работе, б) обратной связи, в) разнообразия навыков, г) идентичности задачи и е) значимости задачи.

Наше сопоставление гибких практик на рисунке 2 показывает частичное совпадение потенциального влияния гибких практик управления проектами (PM) и гибких практик SDA как на автономность работы, так и на обратную связь. Влияние использования гибких практик управления проектами (PM) и гибких практик SDA, вероятно, положительно усилит воздействие каждой из этих практик. Хотя это ещё не проверено, специалисты по гибким практикам давно утверждают, что влияние использования нескольких практик не просто аддитивно, а мультипликативно (например, Beck, 2000; Highsmith, 2002); то есть, что несколько практик дают больший эффект, чем сумма их отдельных частей. Например, автоматизированное тестирование и непрерывная интеграция, вероятно, усиливают влияние итеративной поставки, что позволяет ADT быстро корректировать курс и менять направление по мере необходимости. В связи с этим мы выдвигаем гипотезу:

H4a-b: Существует положительный эффект взаимодействия между степенью использования гибких методов управления проектами и степенью использования гибких методов планирования и разработки решений (SDA) по а) автономности работы и б) обратной связи.

Кроме того, мы предполагаем, что гибкие практики оказывают прямое влияние на удовлетворенность работой. Сторонники гибких методологий сообщают, и некоторые исследования подтверждают, что ADT имеют более высокую степень взаимодействия с клиентом и более высокую мотивацию разработчиков (например, McHugh et al., 2010). Исследования также показали, что ADT предоставляют более качественное программное обеспечение (Maruping et al., 2009a), имеют более короткие сроки проектов и более высокие показатели успешности проектов, чем традиционные командные методы (West et al., 2010). Agile-команды работают итеративно и ставят краткосрочные цели для достижения работающего и поставляемого программного обеспечения каждые несколько недель (Highsmith, 2002). Исследования рассматривают удовлетворенность работой как продукт вознаграждений (материальных и нематериальных), получаемых за производительность (например, Naylor, Pritchard, & Ilgen, 1980; Vroom, 1964). Члены Agile-команды могут ежедневно оценивать свою индивидуальную производительность, используя методы обратной связи, такие как модульное тестирование, непрерывная интеграция и *burndown*. Кроме того, ADT оценивают свою производительность в конце каждого рабочего цикла и получают обратную связь от клиентов о качестве и соответствии своей работы (Beck, 2000; McHugh et al., 2010; Schwaber & Beedle, 2002). Сокращение цикла оценки производительности должно привести к повышению уровня удовлетворенности, поскольку оно открывает возможности для признания успеха и постоянного совершенствования (Burke et al., 2006). В связи с этим мы выдвигаем гипотезу:

H5: Степень использования гибких методов управления проектами положительно влияет на удовлетворенность работой.

H6: Степень использования гибких методов SDA положительно влияет на удовлетворенность работой.

В разделе 4 мы представляем подробности нашего исследования, в ходе которого были проверены эти гипотезы.

4 Метод исследования

Чтобы оценить нашу исследовательскую модель, мы собрали данные опроса в июне 2013 года у 252 респондентов, которые работали как специалисты по разработке программного обеспечения. Мы набрали нашу выборку с помощью Etranel, компании по сбору данных, специализирующейся на подборе ИТ-специалистов с помощью интернет-опросов. В состав группы вошли ИТ-специалисты, отобранные Etranel. Многие исследования в области информационных технологий успешно собирали и использовали данные подобным образом (например, Angst & Agarwal, 2009; Ayuagari, Grover, & Purvis, 2011; Benlian, Titah, & Hess, 2012). Компания предоставила 1098 потенциальных респондентов. Мы спросили каждого потенциального респондента, входят ли они в настоящее время в команду разработчиков программного обеспечения и какую роль в ней играют. Кроме того, чтобы убедиться, что у наших респондентов достаточно опыта и знаний для формирования представления о своей текущей работе, мы попросили их сообщить об общем опыте работы в сфере ИТ и о том, какой опыт они имеют в своей текущей организации³.

На основании ответов на эти контрольные вопросы мы исключили 441 потенциального респондента и включили в опрос оставшихся 657 респондентов.

В качестве дополнительной проверки качества мы разместили вопросы «контроля качества» в нескольких точках опроса, чтобы обеспечить вовлеченность респондентов. Если респонденты отвечали на вопрос о качестве ответа некорректно, мы немедленно прекращали опрос и удаляли их ответы. В результате мы дисквалифицировали 351 респондента, начавших опрос. Наша анкета была длинной, поэтому, чтобы убедиться, что в анализ включены только респонденты, которые тщательно обдумывали каждый вопрос, мы отклонили 51 респондента, заполнивших анкету менее чем за 10 минут⁴. В результате 255 респондентов прошли опрос полностью, ответили на все вопросы о качестве ответа правильно и потратили на заполнение анкеты не менее 10 минут.

Наконец, мы проанализировали эти ответы и исключили ещё трёх респондентов, которые ответили на все вопросы одинаково или указали в открытом вопросе, что они не соответствуют критериям. Мы включили данные оставшихся 252 респондентов в исследование. Из 252 респондентов 125 указали, что работали в командах гибкой разработки, а 127 — нет. Из 252 респондентов 80,7% были мужчинами.

Этот процент согласуется с данными анализа гендерного соотношения в командах разработчиков программного обеспечения (James, 2010). В Приложении A2 выборка разбита по ролям в команде.

По возможности, в опрос были включены шкалы, адаптированные из предыдущих исследований. Для оценки использования гибких методов мы разработали новый набор пунктов. В таблице A1 представлены источники пунктов и инструмент, использованный нами для данного опроса.

5 Анализ модели

Мы проанализировали нашу теоретическую модель, используя метод частичных наименьших квадратов (PLS) с помощью SmartPLS (Ringle, Wende, & Will, 2005). Моделирование структурных уравнений методом частичных наименьших квадратов (PLS-SEM) является подходящим методом анализа для нашей модели, поскольку она относительно сложна, включает эффекты взаимодействия и моделирует гибкие методы в виде агрегированных конструкторов второго порядка (Polites et al., 2012). Мы провели несколько тестов для валидации нашей модели, включая тесты на конвергентную и дискриминантную валидность, а также на общее смещение метода. Результаты представлены в Приложении В. Результаты показывают, что наша модель соответствует или превосходит стандарты валидности (Straub, Boudreau, & Gefen, 2004).

Мы смоделировали agile-конструкции управления проектами (PM) и agile-конструкции управления проектами (SDA) как агрегированные конструкторы, где agile-практики измерялись рефлексивно, а затем факторы второго порядка составлялись из соответствующих практик (Hair, Hult, Ringle, & Sarstedt, 2013). Мы использовали многоэтапный подход для анализа; мы смоделировали подход после иерархической регрессии, основанной на методе, примененном Siponen и Vance (2010). Сначала мы исследовали независимое влияние контрольных переменных (отрицательная аффективность, возраст, стаж работы в организации, пол, образование и общий опыт работы — см. Приложение А), не включая ни один из конструкторов JCM или agile-практик на зависимую переменную удовлетворенности работой. Мы называем эту модель 1. Эти переменные

³ Респонденты должны были быть членами команды разработчиков программного обеспечения, иметь общий опыт работы не менее одного года и проработать в своей текущей организации не менее шести месяцев.

⁴ Позже мы проанализировали наши данные, включив в них 51 респондента. Результаты существенно не изменились.

Объясняло 20% дисперсии удовлетворенности работой. Однако анализ показал, что только негативная аффективность и гендерная принадлежность имели значимую связь с зависимой переменной.

Затем мы проанализировали дополнительные эффекты конструкторов модели характеристик работы (обратная связь, автономия работы, идентичность задачи, значимость задачи и разнообразие навыков), которые мы называем моделью 2. Все эти конструкторы были значимыми (все $p < 0,05$). Дисперсия, объясняемая в модели 2, увеличилась с 20% до 54,2%. Затем мы рассчитали величину различий эффектов между двумя моделями как $f^2 = (R^2_{\text{Model2}} - R^2_{\text{Model1}})/(1 - R^2_{\text{Model2}})$ (Chin, Marcolin, & Newsted, 2003; Qureshi & Compeau, 2009) для проверки значимости этого увеличения. Затем мы провели псевдо-F-тест, умножив размер эффекта (f^2) на $(n - k - 1)$, где n – размер выборки, а k – количество независимых переменных (Mathieson, Peacock, & Chin, 2001).

Размер эффекта был очень большим (.741), что указывает на то, что изменение R^2 было значительным ($F = 181,04$, $p < .01$), что означает, что влияние конструкции модели характеристик работы на дисперсию, объясняемую удовлетворенностью работой (модель 2), было значительно выше, чем влияние, объясняемое моделью 1.

Таблица 4. Сравнение моделей

	Модель 1 только элементы управления	Модель 2 JCM + элементы управления	Модель 3 полная модель
P2	.200	.542	.582
Дельта R2		.342**	.040*
Размер эффекта		.747	.096

**p <.01, *p <.05
Модель характеристик работы JCM

Наконец, мы добавили конструкторы использования agile-практик для завершения полной модели (модель 3) (см. рисунок 2). Полная модель увеличила объясненную дисперсию удовлетворенности работой с 0,542 до 0,582. Величина эффекта составила 0,096, что является небольшим эффектом (Cohen, 1988). Однако, даже несмотря на то, что эффект был небольшим, а изменение объясненной дисперсии составило всего 4%, это изменение оказалось значимым при проведении псевдо-F-теста ($F = 14,55$, $p < 0,05$). Эти тесты подтверждают включение конструкторов использования agile-практик в полную модель. Таблица 4 суммирует этот многоэтапный анализ. Для проверки значимости мы провели бутстреппинг с 252 случаями и 1000 итерациями; на рисунке 4 представлены результаты полной модели.

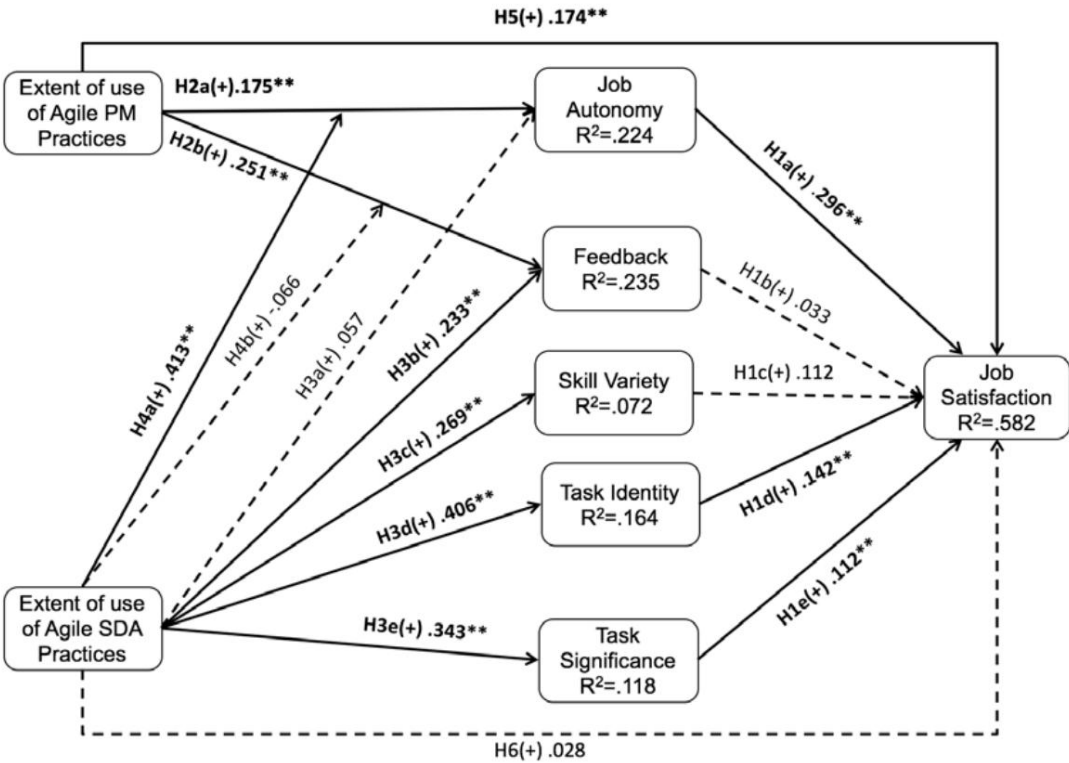


Рисунок 4. Результаты PLS

Интересно, что в то время как все пути от характеристик работы к удовлетворенности работой были значимыми в модели 2, в полной модели пути от обратной связи (0,033) и разнообразия навыков (0,112) не были значимыми. Кроме того, хотя мы предположили, что обе конструкции использования agile-практик оказывают прямое влияние на удовлетворенность работой, только agile-практики управления проектами (0,174) были значимыми при уровне $p < 0,01$. Все наши предполагаемые пути от конструкций использования agile-практик были значимыми, за исключением пути от agile-практик SDA к автономности работы. Взаимодействие между agile-практиками SDA и agile-практиками управления проектами значительно повлияло на автономность работы и не повлияло на обратную связь. Таким образом, наши результаты частично подтверждают H1, H3 и H4 и подтверждают H2 и H5; они не подтверждают H6. Таблица 5 суммирует результаты нашей гипотезы. Кроме того, в нашей окончательной модели отрицательная аффективность была единственной контрольной переменной, которая значимо связана с удовлетворенностью работой. В Разделе 6 мы обсуждаем наши результаты и их значение.

Таблица 5. Сводка результатов гипотез

Гипотеза	Результат	Поддерживается?
H1: Более высокое восприятие характеристик работы положительно связано с удовлетворенностью работой. а) Автономия работы б) Обратная связь с) Разнообразие навыков г) Идентификация задачи д) Значимость задачи	.296** .033 .112 .142** .112**	Да Нет Нет Да Да
H2: Степень использования гибких методов управления проектами положительно влияет на воспринимаемые характеристики работы: а) автономии в работе и б) обратную связь а) Автономия работы б) Обратная связь	.174** .251**	Да Да
H3: Степень использования гибких методов SDA положительно влияет Восприятие характеристик работы: а) Автономия в работе б) Обратная связь с) Разнообразие навыков г) Идентификация задачи д) Значимость задачи	.057 .233** .269** .406** .343**	Нет Да Да Да Да
H4: Положительный эффект взаимодействия между степенью использования гибких методов управления проектами и степенью использования гибких методов управления процессами (SDA) проявляется в следующем: а) Автономность работы б) Обратная связь.	.413** -.066	Да Нет
H5: Степень использования гибких методов управления проектами положительно влияет на удовлетворенность работой.	.174**	Да
H6: Степень использования гибких методов SDA положительно влияет на удовлетворенность работой.	.028	Нет
Примечание: ** - $p < .01$		

6. Обсуждение и выводы

Желая оценить основанные на практике утверждения о положительном влиянии гибких методов на дизайн рабочих мест, мы изучили два исследовательских вопроса:

RQ1: Как степень использования различных гибких практик влияет на удовлетворенность работой?

RQ2: Каковы причинно-следственные связи, посредством которых гибкие практики влияют на удовлетворенность работой?

В ходе перекрестного исследования участников ADT мы обнаружили подтверждение влияния гибких практик на удовлетворенность работой. Во-первых, мы обнаружили, что участники ADT, которые использовали более высокий уровень гибких практик управления проектами, имели более высокий уровень Восприятия удовлетворенности работой (гипотеза 5). Напротив, более высокий уровень использования гибких методов SDA не оказывал прямого влияния на восприятие удовлетворенности работой (гипотеза 6). Во-вторых, мы обнаружили, что степень использования гибких методов влияет на восприятие характеристик работы, хотя не все наши предсказанные пути оказались значимыми (гипотеза 3а). Мы также обнаружили положительное взаимодействие между использованием гибких методов управления проектами и гибких методов SDA в отношении автономности работы, но не в отношении обратной связи. Мы обсудим каждый из этих результатов ниже.

Хотя мы обнаружили общее подтверждение влияния гибких практик на удовлетворенность работой, мы обнаружили, что не все характеристики работы существенно влияли на удовлетворенность работой при использовании гибких методов. В модели 2, которая не включала конструкторы использования гибких методов, все пять характеристик работы существенно влияли на удовлетворенность работой, как и следовало ожидать на основе существующих исследований (например, Ahuja et al., 2007; Hackman et al., 1976; Moore, 2000; Morris & Venkatesh, 2010; Ply et al., 2012). Однако в полной модели (модель 3), которая учитывает использование гибких методов, только три характеристики работы (автономия работы, идентичность задачи и значимость задачи) были значимыми. Мы обнаружили, что обратная связь и разнообразие навыков не имели существенного значения в полной модели.

Возможное объяснение этих результатов связано с неотъемлемыми преимуществами, заложенными в основу разработки гибких практик. Два преимущества внедрения гибких практик – это высокая частота итеративной обратной связи и более широкий спектр навыков. Вполне вероятно, что показатели, ориентированные на обратную связь в гибких практиках, были более значимыми для респондентов, чем более общие показатели, обычно используемые для измерения обратной связи в JCM. Более того, как и предполагалось, тот же эффект может наблюдаться и в отношении конструктора «разнообразие навыков», поскольку как гибкие практики управления проектами (PM), так и гибкие практики управления процессами (SDA) требуют использования более разнообразного набора навыков, чем это может потребоваться в типичной традиционной среде разработки программного обеспечения. Мы считаем, что, хотя характеристики работы являются отличным предиктором удовлетворенности работой, исследования должны разработать дополнительную теорию, касающуюся влияния различных контекстуальных факторов, связанных с информационными технологиями (IS), на формирование восприятия характеристик работы. Данное исследование представляет собой первый шаг к разработке моделей непредвиденных обстоятельств, объясняющих, как структура ИТ-работы делает характеристики работы более или менее значимыми для членов ADT и более широкого круга ИТ-специалистов.

Изучая источники характеристик работы, наша работа продвигает литературу по ИТ-персоналу за пределы ее основания в JCM. В нашем анализе влияние степени использования гибких практик управления проектами было весьма значительным в отношении обратной связи и разнообразия навыков, что подтверждает гипотезы 2a-b. Как и ожидалось, гибкие практики SDA были связаны с более высоким восприятием характеристик работы, специфичных для задачи, включая идентичность задачи, значимость задачи и разнообразие навыков. Степень использования гибких практик SDA также влияла на обратную связь, хотя путь к автономности работы был незначительным. Автономность работы может быть несущественной для гибких команд, поскольку гибкие практики SDA усиливают взаимозависимость работы членов команды. Например, гибкие практики, такие как парное программирование и стандарты кодирования, напрямую фокусируют внимание на ответственности людей за соблюдение норм команды и согласованных процессов. Несмотря на отсутствие каких-либо данных, полученные результаты подтверждают утверждение сторонников о том, что использование гибких методов улучшает восприятие характеристик работы (McHugh & Lang, 2011; Tessem & Maurer, 2007). Таким образом, данная работа наглядно демонстрирует, что такие практики разработки программного обеспечения, как гибкие методы, влияют на восприятие своей работы членами ADT.

Более того, наше исследование предполагает, что наборы практик формируют основные характеристики работы. Мы обнаружили, что гибкие практики управления проектами (PM) и разработки программного обеспечения (SDA) взаимодействовали, положительно влияя на автономность работы (гипотеза 4a). Влияние взаимодействия на автономность работы отражает то, что способность итеративно управлять и менять направление (что отражено в гибких практиках управления проектами) вероятно зависит от использования дополнительных подходов к разработке программного обеспечения. Сторонники Agile утверждают, что создание полного дизайна заранее, вероятно, приведет к пустой трате ресурсов, поскольку понимание бизнес-проблемы со временем меняется. Однако такой подход может привести к необходимости многократного уточнения и потенциального добавления и удаления больших фрагментов кода. Без гибких практик SDA, таких как модульное тестирование и автоматизированные сборки, влияние крупномасштабных изменений повлияет на способность команды реализовать преимущества итеративной разработки. Таким образом, наши результаты подтверждают давно обсуждаемое утверждение сторонников гибких методологий о том, что гибкие методы при совместном использовании оказывают большее влияние, чем сумма их частей (например, Бек, 2000; Хайсмит, 2002).

Мы обнаружили, что взаимодействие гибких концепций PM и agile SDA не всегда влияло на воспринимаемые характеристики работы. Хотя гибкие SDA и agile PM напрямую влияли на обратную связь, эффект взаимодействия был незначимым (гипотеза 4b). Этот результат может быть связан с различиями в практиках обратной связи в двух конструкторах. Мы сопоставили гибкие практики PM с обратной связью через конструкции JDP, такие как установление отношений с клиентами и открытие каналов обратной связи, в то время как гибкие практики SDA сопоставили только через открытие каналов обратной связи. Члены ADT могут воспринимать обратную связь, генерируемую гибкими практиками PM, такими как стендап-встречи и ретроспективы, как в основном обратную связь, генерируемую человеком. Напротив, они могут воспринимать обратную связь, генерируемую гибкими практиками SDA, такими как автоматизированное тестирование и непрерывная интеграция, как обратную связь, генерируемую машиной. Хотя взаимодействие гибких методов управления проектами (PM) и гибкой методологии SDA не повлияло на обратную связь, в будущих исследованиях необходимо изучить, влияет ли разница в этих методах на другие результаты на рабочем месте, такие как профессиональные вознаграждения, или на более часто исследуемые результаты, такие как удовлетворенность работой и текучесть кадров в ADT.

Наши результаты подчеркивают, что гибкие методы управления проектами (PM) и методы управления проектами (SDA) по-разному влияют на восприятие сотрудниками своего рабочего места. Переходя к Гипотезе 5, мы обнаружили прямую связь между уровнем использования гибких методов управления проектами (PM) и удовлетворенностью работой. Гибкие методы управления проектами (PM) направлены на организацию для быстрого и многократного достижения результатов.

программное обеспечение. Поскольку регулярная и повторяющаяся реакция на сигналы окружающей среды имеет решающее значение для оценки успешности команды (Burke et al., 2006), использование гибких методов управления проектами, вероятно, приведет к тому, что члены команды будут считать свою работу успешной и, следовательно, удовлетворяющей. Этот вывод согласуется с ранее опубликованными данными, в которых методы координации описываются как влияющие на удовлетворенность работой (например, McHugh & Lang, 2011). Напротив, наши результаты не подтверждают Гипотезу 6, поскольку уровень использования гибких методов SDA не влиял напрямую на восприятие удовлетворенности работой. Сторонники гибких методов утверждают, что некоторые методы поддерживают использование других методов. Вместо того, чтобы оказывать прямой эффект, гибкие методы SDA могут поддерживать способность команды к итеративной поставке. Если бы это было так, то гибкие методы SDA оказывали бы косвенное влияние на удовлетворенность работой.

6.1 Тесты на медиацию

Учитывая полученные нами результаты, мы провели ретроспективный анализ, чтобы выяснить, были ли наши прямые эффекты частично (agile PM) или полностью (agile SDA) опосредованы восприятием характеристик работы. Используя, казалось бы, не связанный с этим пакет регрессии (sureg) в Stata 13, мы проверили наличие множественной медиации, как это сделали Пречер и Хейс (2008).

Рекомендуем. Мы протестировали каждый косвенный эффект пути по отдельности, а затем объединили эффекты agile PM и agile SDA для оценки общего косвенного эффекта каждой конструкции. Поскольку косвенные эффекты обычно асимметричны и имеют высокий эксцесс, р-значения для проверки значимости не являются надежными. Для определения значимости мы использовали бутстрэппинг (5000 репликаций). Результаты представлены в таблице 6. Мы не обнаружили опосредующего эффекта agile PM и обнаружили небольшой полностью опосредованный эффект (0,091*) agile SDA. Следовательно, полный эффект использования agile PM-практик на удовлетворенность работой был прямым, в то время как полный эффект использования agile SDA-практик был полностью опосредован конструктами JCM, такими как автономия работы, значимость задачи и идентичность задачи.

Таблица 6. Эффекты множественного посредничества

Agile PM -> автономия в работе -> удовлетворенность работой	-.003
Agile PM -> обратная связь -> удовлетворенность работой	.000
Общий эффект посредничества Agile PM	-.003
Agile SDA -> автономия работы -> удовлетворенность работой	.024*
Agile SDA -> обратная связь -> удовлетворенность работой	.000
Agile SDA -> значимость задачи -> удовлетворенность работой	.043*
Agile SDA -> идентификация задачи -> удовлетворенность работой	.034*
Agile SDA -> разнообразие навыков -> удовлетворенность работой	-.010
Общий эффект посредничества Agile SDA	.091*
Примечание: *p<.05	

Эту разницу в прямом и косвенном эффекте agile-концепций управления проектами (PM) и SDA можно объяснить тем, что agile-практики SDA в большей степени ориентированы на индивидуальные задачи, в то время как agile-практики управления проектами (PM) дают членам команды ощущение контроля над широким набором характеристик проекта. Это различие важно.

поскольку JCM описывает контроль отдельных лиц над своими индивидуальными задачами и контекстом, в то время как гибкие методы управления проектами дают отдельным лицам право голоса в более широком контексте на уровне команды и проекта.

6.2 Значение для теории и будущих исследований

Связывая гибкие методы разработки с воспринимаемыми характеристиками работы и удовлетворенностью, наши результаты обладают потенциалом для информирования будущих исследований. Во-первых, в то время как исследования изучали влияние технологических характеристик на характеристики работы (например, Lending & Chervany, 2002), наше исследование является первым, в котором конкретная практика разработки ИС (например, использование гибких методов) рассматривается как антецедент характеристик работы. Таким образом, наши результаты открывают новое направление для исследователей, стремящихся разработать теорию управления ИТ-персоналом. В предыдущих исследованиях характеристики работы часто рассматривались как экзогенные и, как правило, изучались ИТ-контексты с использованием модели, описанной Хакманом и Олдхэмом (1980) (например, Ahuja et al., 2007; Moore, 2000; Thatcher et al., 2002), или модель Хакмана и Олдхэма использовалась в качестве линзы для изучения изменений в восприятии, вызванных внедрением ИТ (например, Morris & Venkatesh, 2010).

Однако наше исследование показывает, что организационные и процессные конструкты, связанные с ИС, могут влиять на формирование восприятия характеристик работы. Дальнейшие исследования должны изучить влияние других конструктов, связанных с ИС, на формирование восприятия конструктов JCM.

Кроме того, различая гибкие методы управления проектами (PM) и гибкой методологии разработки программного обеспечения (SDA) и их влияние, мы иллюстрируем важность более детального подхода к практике разработки программного обеспечения. Этот более детальный подход предполагает, что вместо того, чтобы рассматривать «гибкие методы разработки» как монолитную концепцию, в будущих исследованиях необходимо тщательно изучить фокус и структуру, предлагаемые методом или практикой, в связи с удовлетворенностью работой и другими восприятиями. Например, наши результаты показывают, что некоторые agile-практики напрямую влияют на отдельного человека, а другие – на команду в целом. Кроме того, наши результаты указывают на более сложную структуру влияния agile-практик на результаты работы, где одни практики поддерживают другие. Сторонники agile-практиков утверждают, что применение нескольких agile-практик имеет комплексный эффект и способствует внедрению других практик (Beck, 2000). Исследования также выявили схожие нелинейные эффекты применения нескольких практик в контексте управления персоналом.

(Ферратт, Агарвал, Браун и Мур, 2005). Следовательно, понимание того, работает ли номологическая сеть конструктов в контексте использования гибких методов, стало бы значительным вкладом. Такие исследования важны, поскольку они могут помочь руководителям правильно адаптировать гибкие методы к проектам, чтобы они оказывали более положительное влияние как на качество продукта, так и на удовлетворенность сотрудников.

Более того, наши результаты закладывают основу для исследований, изучающих гибкие методы управления проектами (PM) и гибкие методы планирования и разработки (SDA) в контексте восприятия и поведения на работе, таких как восприятие перегрузки, выгорание, расширение прав и возможностей, производительность и текучесть кадров. Такие исследования должны учитывать не только влияние на организацию, но и сравнение гибких методов с традиционными с течением времени (например, на протяжении всего проекта или карьеры специалиста). Например, по мере того, как участник ADT накапливает опыт работы над различными проектами и применения различных гибких методов, он, вероятно, будет лучше понимать влияние гибких методов. Поскольку в большинстве предыдущих работ гибкая разработка рассматривается как монолитная концепция, исследования практически не затрагивали восприятие применения гибких методов, результаты успешности проектов, сопутствующие изменения характеристик работы и влияние этих факторов на членов ADT с течением времени. Кроме того, будущие исследования могут изучить взаимосвязь между успехом гибких проектов и удовлетворенностью работой, поскольку влияние командной работы и индивидуальной удовлетворенности работой не является независимым (например, Sox, 2003).

Помимо гибких методов, исследователи должны оценить влияние успеха проекта на представления членов ADT о последствиях применения практик ADT. Например, гибкие методы открывают каналы обратной связи посредством приёмочных тестов, которые пишут представители клиента или владельцы продукта⁵ (Matook & Maruping, 2014). Сотрудники, занимающие эту должность, должны одобрять или не одобрять реализацию конкретных функций командой гибкой разработки. и команда получает обратную связь о том, насколько хорошо ее программное обеспечение соответствует спецификациям клиента. Тем не менее, хотя исследования показали, что такие изменения положительно влияют на воспринимаемые характеристики работы, мы мало знаем о том, как такие изменения влияют на глобальное восприятие членами ADT своей работы, своих проектов или организации-работодателя в целом. Интуитивно понятно, что члены ADT, которые считают, что использование agile-практик связано с более высоким успехом проектов, будут считать использование этих практик положительным событием. И наоборот, если члены ADT считают, что их проекты не успешны, они могут посчитать использование agile-практик изнурительным с потенциальным снижением удовлетворенности работой. Исследования должны изучить взаимосвязь между agile-практиками PM и agile-практиками SDA и восприятием сотрудниками ADT успешности проекта и результатами, связанными с работой и организацией, такими как выгорание или приверженность организации.

Хотя мы концептуализировали это исследование на индивидуальном уровне, существуют широкие возможности для многоуровневого изучения разработки программного обеспечения и гибких методологий. Ввиду высокой взаимозависимости командной разработки программного обеспечения, исследование должно быть направлено на изучение нашей теории на уровне команд или с использованием многоуровневого подхода к организациям. Такое исследование может дать дополнительную информацию о различиях в удовлетворенности работой внутри команд и между ними, включив в него несколько респондентов из одной команды. Например, изучение взаимодействия между промышленностью, гибкими методами работы и методами лидерства может дать более глубокое понимание удовлетворенности работой членов ADT и других представлений о рабочем месте.

Наконец, мы признаем, что для полного понимания влияния гибких методов управления проектами (PM) и разработки программного обеспечения (SDA) на членов гибких команд разработки (Behson, 2012) необходимо выйти за рамки модели характеристик работы. Agile-команды вносят значительный вклад в формирование требований своей работы (например, итеративное планирование, ретроспективы). Таким образом, модель нагрузки на работу Карасека (1979), утверждающая, что баланс между полномочиями и ответственностью обеспечивает сотруднику свободу действий, может предоставить новый взгляд на исследование влияния гибких методов на производительность команды и удовлетворенность работой. Учитывая, что ADT намеренно самоуправяемы и часто демонстрируют высокую производительность, модель нагрузки на работу может предоставить возможность концептуализировать новый теоретический подход к удовлетворенности работой в ADT. Кроме того, JCM — лишь один из небольших компонентов исследований дизайна работы и его результатов (например, Grant, Fried, & Juillerat, 2011; Joseph et al., 2007). Дальнейшие исследования гибких методов

⁵ Благодарим анонимного рецензента за это предложение.

Исследуем влияние гибких моделей управления проектами (PM) и управления процессами (SDA) на ряд дополнительных показателей, таких как обучение, производительность и стресс, а также взаимодействие с другими моделями, такими как взаимозависимость. Мы по-прежнему предстоит еще многому научиться относительно влияния гибких методов как катализаторов перестройки рабочих процессов.

6.3 Значения для практики

Наше исследование имеет ряд практических последствий. Во-первых, оно подтверждает ключевой аргумент сторонников Agile (Beck, 2000), демонстрируя, что широкое внедрение Agile-практик, как в сфере управления, так и в сфере задач, приводит к нелинейному влиянию на организацию. Этот нелинейный эффект предполагает, что руководству следует предлагать стимулы и поощрять команды выходить за рамки экспериментального внедрения Agile и внедрять более гибкие методы разработки программного обеспечения. Несмотря на то, что отраслевые исследования показали широкое распространение Agile-практик, во многих, если не в большинстве подобных проектов, Agile-практики используются относительно редко (Conboy & Fitzgerald, 2010).

Однако наше исследование также показывает, что «больше» не обязательно означает «лучше», по крайней мере, когда речь идет о влиянии гибких методов управления проектами (PM) и гибкой методологии разработки (SDA) на удовлетворенность работой. Наши результаты показывают, что влияние гибких методов на удовлетворенность работой наиболее сильно при использовании как гибких методов управления проектами (PM), так и гибких методов разработки (SDA). Прямое влияние гибких практик управления проектами на производительность труда заметно, но оно дополняется прямым влиянием как гибких практик управления проектами, так и гибких SDA на восприятие работы. Гибкие конструкции управления проектами в первую очередь влияют на управленческую сторону JCM (автономность работы и обратная связь), в то время как гибкие конструкции SDA в первую очередь влияют на сторону задач JCM (разнообразие навыков, идентичность задачи и значимость задачи). Для менеджеров этот вывод предполагает, что внедрение только одного типа гибких практик (SDA или PM) окажет меньшее влияние на удовлетворенность работой, чем внедрение обоих типов практик. Поэтому руководство должно осознавать синергетический эффект, достигаемый от поддержки и поощрения гибких SDA и гибких практик управления проектами, особенно на восприятие автономии работы, которое является сильным предиктором удовлетворенности работой.

Наконец, наши результаты должны побудить практиков Agile говорить об Agile-разработке как о чем-то большем, чем просто метод разработки ПО, а как о способе организации и проектирования работы таким образом, чтобы она повышала удовлетворенность работой. Поскольку удовлетворенность является предиктором текучести кадров, которая снижает производительность команды (Argote, Insko, Yovetich, & Romero, 1995), руководство должно стремиться к повышению удовлетворенности работой. Практики Agile могут утверждать, что использование Agile-практик меняет работу команд разработки ПО таким образом, что повышает позитивное восприятие членами команды характеристик работы. Подчеркивая различие между конкретными практиками Agile PM и Agile SDA и их влиянием на характеристики работы, наша работа предоставляет практикам возможности обосновать включение или исключение определенных Agile-практик при организации новых проектов или устранении недостатков в существующих проектах.

7 ограничений

Это исследование имеет ряд ограничений. Во-первых, что касается обобщения наших результатов, мы сделали следующее:

Выборка из специалистов по информационным технологиям, работающих в США. В будущих исследованиях, возможно, будут учтены культурные, географические и другие различия, чтобы расширить возможности обобщения данного исследования в различных контекстах. Кроме того, мы не изучали аутсорсинг, офшоринг и статус работы (т.е. работа по контракту или постоянная работа) членов ADT. Во-вторых, данные были предоставлены самими участниками, а метод сбора данных был поперечным.

В Приложении В описаны два теста, которые показывают, что дисперсия общего метода не повлияла на наши результаты. В будущих исследованиях необходимо собрать данные лонгитюдного анализа для более строгой проверки нашей теории.

Наконец, хотя исследования показали, что характеристики рабочего места являются важным фактором, влияющим на удовлетворенность работой (McKnight, Phillips, & Hardgrave, 2009), мы сосредоточились на влиянии методов гибкой разработки на восприятие характеристик работы. По этой причине, а также в связи с тем, что инструмент нашего опроса и без того был чрезвычайно длинным, мы не включали характеристики рабочего места в нашу модель. Однако можно предположить, что как гибкие методы управления проектами (PM), так и гибкие методы управления мобильностью (SDA) могут влиять на такие характеристики рабочего места, как обмен информацией и доверие к высшему руководству (McKnight et al., 2009). Эта область представляет собой потенциальную возможность для будущих исследований.

8 Заключение

Мы разработали модель влияния практик гибкой разработки на удовлетворенность работой членов ADT. Мы обнаружили неоднозначное подтверждение наших гипотез о влиянии использования практик гибкой разработки на удовлетворенность работой и на восприятие характеристик работы. Данное исследование показывает, что организационные и процессные конструкты, связанные с информационными системами, связаны с восприятием характеристик работы. Конструкты Agile SDA в первую очередь связаны с аспектом задач в характеристиках работы, в то время как конструкты Agile PM в первую очередь связаны с

к управленческому аспекту характеристик работы. Проницательные руководители должны поддерживать и поощрять как гибкие методы SDA, так и гибкие методы управления проектами, чтобы максимально повысить удовлетворенность работой членов АДТ.

Наша модель вносит вклад в литературу по гибкой разработке программного обеспечения и ИТ-персоналу, предоставляя теоретический взгляд, иллюстрирующий пути, посредством которых использование гибких методов влияет на удовлетворенность работой, а также предоставляет доказательства того, как принципы проектирования рабочих мест Хэкмана и Олдхэма (1980) влияют на восприятие характеристик работы.

Ссылки

- Ахуджа, М.К., Чудоба, К.М., Кацмар, К.Дж., Макнайт, Д.Х. и Джордж, Дж.Ф. (2007). ИТ-специалисты, работающие в дороге: баланс между конфликтами между работой и семьей, автономностью в работе и перегрузкой для снижения текучести кадров. *MIS Quarterly*, 31(1), 1–17.
- Андерсон, Дж. Д. (2004). Гибкое управление разработкой программного обеспечения: применение теории ограничений для достижения бизнес-результатов. Нью-Йорк: Prentice Hall.
- Энг, С. и Слотер, С.А. (2001). Результаты работы и структура работы при контрактной и постоянной работе. Системные специалисты в группах разработки программного обеспечения. *MIS Quarterly*, 25(3), 321–350.
- Андерсон, Дж. Д. (2004). Гибкое управление разработкой программного обеспечения: применение теории ограничений для достижения бизнес-результатов. Нью-Йорк: Prentice Hall.
- Энг, С. и Слотер, С.А. (2001). Результаты работы и структура работы при контрактной и постоянной работе. Системные специалисты в группах разработки программного обеспечения. *MIS Quarterly*, 25(3), 321–350.
- Ангст, К.М. и Агарвал, Р. (2009). Внедрение электронных медицинских карт в условиях конфиденциальности: модель вероятности разработки и индивидуальное убеждение. *MIS Quarterly*, 33(2), 339–370.
- Арготе, Л., Инско, К.А., Йоветич, Н. и Ромеро, А.А. (1995). Кривые группового обучения: влияние текучести кадров и сложности задач на групповую эффективность. *Журнал прикладной социальной психологии*, 25(6), 512–529.
- Ауэр, К. и Миллер, Р. (2002). Применение экстремального программирования. Нью-Йорк: Addison-Wesley.
- Айягари, Р., Гровер, В. и Пёрвис, Р. (2011). Техностресс: технологические предпосылки и последствия. *Ежеквартальный журнал МИС*, 35(4), 831–858.
- Балиджепалли, В., Махапатра, Р.К., Нерур, С.П. и Прайс, К. (2009). Две головы лучше, чем одна для разработки программного обеспечения? Парадокс производительности парного программирования. *Management Information Systems Quarterly*, 33(1), 91–118.
- Бек. (2003). Разработка через тестирование: на примере. Нью-Йорк: Addison-Wesley.
- Бек, К. (2000). Экстремальное программирование: Примите перемены. Нью-Йорк: Эддисон-Уэсли.
- Бехсон, С.Дж. (2012). Использование относительных весов для повторного анализа исследований модели характеристик работы. *Журнал организационной психологии*, 12(2), 71–81.
- Бенлиан, А., Титах, Р. и Хесс, Т. (2012). Различное влияние рекомендаций поставщиков и отзывов потребителей на транзакции в электронной коммерции: экспериментальное исследование. *Журнал систем управленческой информации*, 29(1), 237–272.
- Бём, Б. и Тернер, Р. (2004). Баланс между гибкостью и дисциплиной: руководство для растерянных. Бостон, Массачусетс: Эддисон Уэсли.
- Бём, Б. В. (1984). Экономика программной инженерии. *Труды IEEE по программной инженерии*, 10(1), 4–21.
- Берк, К., Стэгл, К., Салас, Э., Пирс, Л. и Кендалл, Д. (2006). Понимание командной адаптации: концептуальный анализ и модель. *Журнал прикладной психологии*, 91(6), 1189–1207.
- Берк, Р. Дж. и Грингласс, Э. (1995). Лонгитюдное исследование психологического выгорания у учителей. *Отношения*, 48(2), 187–202.
- Чин, У. В. (1998). Комментарий: Проблемы и мнения о моделировании структурными уравнениями. *MIS Quarterly*, 22(1), vii–xvi.
- Чин, У. В., Марколин, Б. Л. и Ньюстед, П. Р. (2003). Метод моделирования латентных переменных методом наименьших квадратов для измерения эффектов взаимодействия: результаты исследования методом Монте-Карло и исследования эмоций и восприятия электронной почты. *Information Systems Research*, 14(2), 189–217.
- Чин, У. В., Тэтчер, Дж. Б. и Райт, Р. Т. (2012). Оценка смещения общего метода: проблемы с Методика ULMC. *MIS Quarterly*, 36(3), 1003–1019.
- Кокберн, А. (2001). Гибкая разработка программного обеспечения. Бостон, Массачусетс: Addison-Wesley.

- Кокберн и Уильямс, Л. (2001). Издержки и преимущества парного программирования. В книге: G. Succi & M. Marchesi (ред.), « Исследование экстремального программирования» (стр. 223–248). Нью-Йорк: Addison-Wesley.
- Коэн, Дж. (1988). Статистический анализ мощности для поведенческих наук. Сан-Диего, Калифорния: Psychology Press.
- Конбой, К. и Фицджеральд, Б. (2010). Характеристики методов и разработчиков для эффективной адаптации гибких методов: исследование мнений экспертов по опыту разработки (XP). ACM Transactions on Software Engineering and Methodology, 20(1).
- Кордери, Дж. Л., Моррисон, Д., Райт, Б. М., и Уолл, Т. Д. (2010). Влияние автономии и неопределенности задач на эффективность команды: лонгитюдное полевое исследование. Журнал организационного поведения, 31(2-3), 240–258.
- Кокс, К.Б. (2003). Влияние внутриличностных, внутригрупповых и межгрупповых конфликтов на эффективность работы команды и удовлетворенность работой. Nursing Administration Quarterly, 27(2), 153-163.
- Дерби, Э., Ларсен, Д. и Швабер, К. (2006). Agile-ретроспективы: как сделать хорошие команды великими. Роли, Северная Каролина: Pragmatic Bookshelf.
- Дойч, М. (1960). Влияние мотивационной ориентации на доверие и подозрение. Человеческие отношения, 13, 123-139.
- Дингер, М., Тэтчер, Дж. Б., Тредуэй, Д., Степина, Л. и Бреланд, Дж. (2015). Имеет ли значение профессионализм в сфере ИТ? Эмпирическое исследование ИТ-специалистов. Журнал Ассоциации информационных систем, 16(4), 281-313.
- Дюваль, П.М., Матьяс, С. и Гловер, А. (2007). Непрерывная интеграция: повышение качества программного обеспечения и снижение рисков. Нью-Йорк: Addison-Wesley.
- Ферратт, Т.В., Агарвал, Р., Браун, К.В., и Мур, Дж.Э. (2005). Конфигурации управления ИТ-ресурсами и текучесть кадров в ИТ-сфере: теоретический синтез и эмпирический анализ. Исследования информационных систем, 16(3), 237-255.
- Форнелл, К. и Ларкер, Д.Ф. (1981). Оценка моделей структурных уравнений с ненаблюдаемыми переменными и Ошибка измерения. Журнал маркетинговых исследований, 18(1), 39-50.
- Фаулер, М. (1999). Рефакторинг. Улучшение дизайна существующего кода. Реддинг, Массачусетс: Эддисон Уэсли Лонгман.
- Фаулер, М. (2006). Непрерывная интеграция. Источник: <http://www.martinfowler.com/articles/continuous интеграция.html>
- Фаулер, М. и Хайсмит, Дж. (2001). Манифест Agile. Разработка программного обеспечения, 9(8), 28-35.
- Гефен, Д. и Штрауб, Д. (2005). Практическое руководство по факторной валидности с использованием PLS-Graph: учебное пособие и аннотированный пример. Сообщения Ассоциации информационных систем, 16, 99–109.
- Грант, А.М., Фрид, Ю. и Джульерат, Т. (2011). Работа имеет значение: проектирование рабочих мест в классическом и современном аспектах. В книге С. Зедека (ред.), Справочник Американской психологической ассоциации (АПА) по промышленной и организационной психологии. Вашингтон, округ Колумбия: Американская психологическая ассоциация.
- Гриффет, Р. В., Хом, П. В. и Гертнер, С. (2000). Метаанализ факторов, лежащих в основе текучести кадров, и их коррелятов: обновление, модераторы и исследовательские выводы для следующего тысячелетия. Журнал управления, 26(3), 463-488.
- Хэксман, Дж. и Олдрич, Г. (1980). Перепроектирование труда (ред. Э. Шейн и Р. Бекхард). Нью-Йорк: Addison Wesley.
- Хакман, Дж. Р., Брусс, К. Р., и Вайс, Дж. А. (1976). Взаимодействие разработки задач и стратегий групповой деятельности в определении эффективности группы. Организационное поведение и деятельность человека, 16(2), 350-365.
- Хэйр, Дж., Халт, Г.Т., Рингл, К. и Сарстедт, М. (2013). Введение в структурное уравнение с использованием метода наименьших квадратов. моделирование (PLS-SEM). Таузенд-Оукс, Калифорния: Sage.
- Харрис, М., Коллинз, Р. и Хевнер, А. (2009). Управление разработкой гибкого программного обеспечения в условиях неопределенности. Исследования информационных систем, 20(3), 400-419.

- Харрисон, Д.А., Ньюман, Д.А., и Рот, П.Л. (2006). Насколько важны отношения к работе? Метааналитическое сравнение интегративных поведенческих результатов и временных последовательностей. *Academy of Management Journal*, 49(2), 305-325.
- Хайсмит, Дж. (2002). Экосистемы гибкой разработки программного обеспечения. Бостон: Addison Wesley.
- Джеймс, Дж. (2010). Гендерный разрыв в сфере ИТ: где женщины-программисты? TechRepublic. Источник: <http://www.techrepublic.com/blog/software-engineer/it-gender-gap-where-are-the-female-programmers/>
- Джозеф, Д., Нг, К.-Й., Ко, К., и Анг, С. (2007). Текучесть кадров в сфере информационных технологий: Обзор повествования, метааналитическое структурное моделирование уравнений и разработка модели. *MIS Quarterly*, 31(3), 547-577.
- Карасек Р.А., младший (1979). Требования к работе, свобода принятия решений и умственное напряжение: влияние на работу перепроектирование. *Журнал «Административная наука»*, 24(2), 285-308.
- Кюстер, Й., Гшвинд, Т. и Циммерманн, О. (2009). Инкрементальная разработка цепочек преобразования моделей с использованием автоматизированного тестирования. В книге под ред. А. Шурра и Б. Селича «MODELS 2009» (т. 5795, с. 733-2009). 747). Гейдельберг: Springer.
- Лейман, Л., Уильямс, Л. и Каннингем, Л. (2004). Изучение экстремального программирования в контексте: отраслевой кейс (стр. 32-41). Доклад, представленный на конференции по гибкой разработке.
- Лендинг, Д. и Червани, Н. Л. (2002). Использование инструментов кейса и проектирование работы: объяснение с точки зрения ограничений и гибкости. *Журнал компьютерных информационных систем*, 43(1), 81-90.
- Левики, Р. Дж. и Банкер, Б. Б. (1995). Доверие в отношениях: модель развития и упадка. В ВВ Банкер и Дж. З. Рубин (ред.), *Конфликт, сотрудничество и правосудие: эссе, вдохновлённые работами Мортон Дойча*. Сан-Франциско: Jossey-Bass.
- Льюис, Дж. Д. и Вайгерт, А. (1985). Доверие как социальная реальность. *Social Forces*, 63(4), 967-985.
- Лян, Х., Сараф, Н., Ху, Ц. и Сюэ, И. (2007). Ассимиляция корпоративных систем: влияние институционального давления и посредническая роль высшего руководства. *MIS Quarterly*, 31(1), 59-87.
- Маннaro, К., Мелис, М. и Марчези, М. (2004). Эмпирический анализ удовлетворенности ИТ-сотрудников, сравнивающий методы XP с другими методологиями разработки программного обеспечения. В книге «Экстремальное программирование и гибкие процессы в программной инженерии» (стр. 166-174). Нью-Йорк: Springer.
- Манн, К. и Маурер, Ф. (2005). Исследование влияния скрам-системы на сверхурочную работу и удовлетворенность клиентов. В материалах конференции Agile (стр. 70-79).
- Марупинг, Л., Венкатеш, В. и Агарвал, Р. (2009a). Взгляд теории управления на использование гибкой методологии и изменение требований пользователей. *Information Systems Research*, 20(3), 377-399.
- Марупинг, Л., Чжан, Х. и Венкатеш, В. (2009b). Роль коллективного владения и стандартов кодирования в координации экспертных знаний в командах разработчиков программного обеспечения. *Европейский журнал информационных систем*, 18(4), 355-371.
- Мэтисон, К., Пикок, Э. и Чин, У. В. (2001). Расширение модели принятия технологий: влияние воспринимаемых пользовательских ресурсов. *База данных ACM SigMIS*, 32(3), 86-112.
- Матук, С. и Марупинг, Л.М. (2014). Модель компетенций для представителей заказчика в проектах гибкой разработки программного обеспечения. *MIS Quarterly Executive*, 13(2), 77-95.
- Макхью, К., К. и Лэнг, М. (2011). Использование гибких методов для повышения мотивации участников команд ИТ-проектов. *Скандинавский журнал информационных систем*, 23(2), 85-110.
- Макхью, О., Конбой, К. и Лэнг, М. (2010). Мотивация Agile-команд: пример команд в Ирландии и Швеции. Доклад на 5-м Международном исследовательском семинаре по управлению информационными проектами, предвещающем конференцию ICIS.
- Макхью, О., Конбой, К. и Лэнг, М. (2012). Гибкие методы: влияние на доверие в командах разработчиков программного обеспечения. *Программное обеспечение, IEEE*, 29(3), 71-76.
- Макнайт, Д. Х., Филлипс, Б. и Хардгрэйв, Б. К. (2009). Что больше всего снижает вероятность текучести кадров в ИТ-сфере: характеристики рабочего места или характеристики работы? *Информация и менеджмент*, 46(3), 167-174.

- Мельник, Г. и Маурер, Ф. (2004). Прямое вербальное общение как катализатор гибкого обмена знаниями. Труды конференции IEE по гибкой разработке (стр. 21-31).
- Мельник, Г. и Маурер, Ф. (2006). Сравнительный анализ удовлетворенности работой в командах разработчиков программного обеспечения, использующих и не использующих гибкие методологии. В книге «Экстремальное программирование и гибкие процессы в программной инженерии» (стр. 32-42). Нью-Йорк: Springer.
- Мо, Н., Дингсойр, Т. и Дайба, Т. (2010). Модель командной работы для понимания гибкой команды: пример проекта Scrum. Информационные и программные технологии, 52(5), 480-491.
- Мур, Дж. Э. (2000). Один из путей к текучести кадров: исследование усталости от работы у специалистов в сфере технологий. Ежеквартальный журнал систем управленческой информации, 24(1), 141-168.
- Моррис, М.Г. и Венкатеш, В. (2010). Характеристики работы и удовлетворенность работой: понимание роли внедрения системы планирования ресурсов предприятия. MIS Quarterly, 34(1), 143-161.
- Нам, А.Й., Рао, С.С., Солис-Галван, Л.Е., и Рагу-Натан, Т.С. (2002). Метод Q-сортировки: оценка надежности и конструктивной валидности пунктов анкеты на этапе предварительного тестирования. Журнал современных прикладных статистических методов, 1(1), 114-125.
- Нейлор, Дж. К., Притчард, Р. Д., и Илджен, Д. Р. (1980). Теория поведения в организациях. Нью-Йорк: Академическая пресса.
- Нерур, С., Махапатра, Р.К., и Мангаларадж, Г. (2005). Проблемы перехода на гибкие методологии. Сообщения ACM, 48(5), 72-78.
- Нево, С. и Ченгалур-Смит, И. (2011). Повышение эффективности виртуальных команд разработки программного обеспечения посредством использования гибких методов: пилотное исследование. В материалах Гавайской международной конференции по системным наукам (стр. 1-10).
- Наннали, Дж. К., Бернштейн, И. Х., и Берге, Дж. М. (1967). Психометрическая теория (т. 226). Нью-Йорк: Макгроу-Хилл.
- Палмер, С.П., и Фельсинг, Дж.М. (2002). Практическое руководство по разработке на основе функций. Верхняя Сэддл-Ривер, Нью-Джерси: Прентис Холл.
- Павлоу, П.А., Лян, Х. и Сюэ, И. (2007). Понимание и снижение неопределенности в отношениях онлайн-обмена: перспектива принципала-агента. Management Information Systems Quarterly, 31(1), 105-136.
- Педриц, В., Руссо, Б. и Суччи, Г. (2011). Модель удовлетворенности работой для процессов совместной разработки. Журнал систем и программного обеспечения, 84(5), 739-752.
- Петтер, С., Штрауб, Д. и Рай, А. (2007). Определение формирующих конструктов в исследовании информационных систем. Ежеквартальный журнал МИС, 31(4), 623-656.
- Плай, Дж. К., Мур, Дж. Э., Уильямс, К. К., и Тэтчер, Дж. Б. (2012). Отношение и восприятие сотрудников ИС на разных уровнях зрелости процесса разработки программного обеспечения. MIS Quarterly, 36(2), 601-624.
- Подсакофф, П.М., Маккензи, С.Б., Ли, Дж.Й. и Подсакофф, Н.П. (2003). Распространенные методологические ошибки в поведенческих исследованиях: критический обзор литературы и рекомендуемые меры. Журнал прикладной психологии, 88(5), 879-903.
- Политес, Г.Л., Робертс, Н. и Тэтчер, Дж. (2012). Концептуализация моделей с использованием многомерных конструктов: обзор и рекомендации по их использованию. Европейский журнал информационных систем, 21(1), 22-48.
- Пречер, К. Дж., и Хейс, А. Ф. (2008). Асимптотические и ресемплинговые стратегии для оценки и сравнения косвенные эффекты в моделях с несколькими медиаторами. Методы исследования поведения, 40(3), 879-891.
- Куреши, И. и Компо, Д. (2009). Оценка межгрупповых различий в исследованиях информационных систем: сравнение ковариационного и компонентного SEM. MIS Quarterly, 197-214.
- Рингле, К.М., Венде, С. и Уилл, А. (2005). Smart PLS 2.0. Гамбургский университет. Источник: <http://www.smartpls.de>
- Ратнер, П.С., Хардгрейв, Б.К., и Макнайт, Д.Х. (2008). Эмоциональный диссонанс и информация технологический специалист. MIS Quarterly, 32(3), 635-652.

- Швабер, К. и Бидл, М. (2002). Гибкая разработка программного обеспечения с использованием Scrum. Анпер Сэддл Ривер, Нью-Джерси: Прентис Холл.
- Сипонен, М. и Вэнс, А. (2010). Нейтрализация: новый взгляд на проблему информации о сотрудниках
Нарушения политики безопасности систем. MIS Quarterly, 34(3), 487-502.
- Штрауб, Д., Будро, М.-К. и Гефен, Д. (2004). Руководство по валидации исследований в области позитивизма IS.
Сообщения Ассоциации информационных систем, 13, 380-427.
- Сазерленд, Дж. (2001). Agile может масштабироваться: изобретение и переосмысление Scrum в пяти компаниях. Журнал Cutter IT. 14(12), 5-11.
- Тессем, Б. и Маурер, Ф. (2007). Удовлетворенность работой и мотивация в большой гибкой команде. В книге Г. Конкаса, Э. Дамиани, М. Скотто и Дж. Суччи (редакторы), XP 2007 (т. 4536, с. 54-61). Нью-Йорк: Springer.
- Тэтчер, Дж. Б., Лю, Ю., Степина, Л. П., Гудман, Дж. М. и Тредвей, Д. К. (2006). Текучесть кадров в сфере ИТ: Эмпирическое исследование внутренней мотивации. База данных ACM SIGMIS, 37(2-3), 133-146.
- Тэтчер, Дж. Б., Степина, Л. П., и Бойл, Р. Дж. (2002). Текучесть кадров в сфере информационных технологий: эмпирическое исследование влияния установок, характеристик работы и внешних рынков. Журнал систем управленческой информации, 19(3), 231-261.
- Трипп, Дж. Ф. и Армстронг, Д. Дж. (2014). Исследование взаимосвязи между мотивами внедрения гибких методов в организациях и их адаптацией. В материалах 47-й Гавайской международной конференции по системным наукам (стр. 4799-4806).
- VersionOne. (2011). 6-й ежегодный опрос о состоянии гибкой разработки. Материал взят с сайта www.versionone.com
- фон Хиппель, К., Калокеринос, Э.К. и Генри, Дж.Д. (2013). Угроза стереотипов среди пожилых сотрудников: связь с отношением к работе и намерениями текучести кадров. Психология и старение, 28(1), 17-27.
- Врум, В. Х. (1964). Работа и мотивация. Оксфорд, Англия: Wiley.
- Вайс, Х. М. и Кропанцано, Р. (1996). Теория аффективных событий: теоретическое обсуждение структуры, причин и последствий аффективных переживаний на работе. В книге Б. М. Стоу и Л. Л. Каммингса (ред.), Исследования организационного поведения: ежегодная серия аналитических эссе и критических обзоров (т. 18, стр. 1-75). Нью-Йорк: Elsevier.
- Уэст, Д., Грант, Т., Геруш, М. и Д'Сильва, Д. (2010). Гибкая разработка: массовое внедрение изменило гибкость. Forrester Research.
- Уильямс, Л. Дж., Эдвардс, Дж. Р. и Ванденберг, Р. Дж. (2003). Современные достижения в методах причинно-следственного моделирования для организационных и управленческих исследований. Журнал управления, 29(6), 903-936.
- Райт, Р.Т., Кэмпбелл, Д.Э., Тэтчер, Дж.Б. и Робертс, Н. (2012). Операционализация многомерных конструкций в моделировании структурных уравнений: рекомендации по исследованиям информационных систем. Сообщения Ассоциации информационных систем, 30(1), 367-412.

Приложение А: Инструмент

Для пунктов измерения мы адаптировали некоторые шкалы из предыдущих исследований: мы измеряли удовлетворенность работой и характеристики работы, используя шкалы, слегка модифицированные из шкал Хэкмана и Олдхэма (1980) (Morris & Venkatesh, 2010). Для измерения использования гибких методов мы разработали новый набор пунктов. Участники ADT иногда используют схожие термины для описания своих практик, даже если эти практики не выполняются одинаково. Таким образом, мы разработали шкалу, которая отражала практики в соответствии с литературой по гибким методологиям (например, Beck, 2000; Schwaber & Beedle, 2002). Для разработки этой шкалы один из авторов использовал публикации по гибкой разработке и вклад внешнего исследователя и практика гибких методологий для разработки начального набора пунктов для каждой из 10 интересующих гибких практик. После разработки этих пунктов мы следовали рекомендациям по применению Q-сортировки (Nahm, Rao, Solis-Galvan, & Ragu-Nathan, 2002). Мы привлекли двух дополнительных agile-специалистов для проведения сортировки. Первоначальное согласие по шкалам составило около 60%. Мы обсудили неудовлетворительные результаты с экспертами, проводившими сортировку. По результатам обсуждения мы модифицировали или заменили элементы, а три дополнительных члена agile-команды провели второй раунд сортировки. Каппа Коэна для этого упражнения по сортировке составила более 85%, что свидетельствует об отличном согласии (Nahm et al., 2002). Мы использовали эти элементы для исследования.

Поскольку исследования показали, что удовлетворенность работой является одним из лучших предикторов намерения текучести кадров (Griffeth, Hom, & Gaertner, 2000), мы изучили несколько контрольных переменных, которые часто использовались в исследованиях текучести кадров в ИТ-сфере, и выбрали возраст и стаж работы в организации (Ahuja et al., 2007; Joseph et al., 2007; Moore, 2000), негативную аффективность (Moore, 2000), а также пол и образование (Joseph et al, 2007). Исследования отношения к работе в психологической литературе также использовали пол, стаж работы и возраст в качестве контрольных переменных (von Hippel, Kalokerinos, & Henry, 2013). Кроме того, мы также включили общий опыт работы в качестве контрольной переменной, чтобы учесть возможность того, что кульминация общего опыта работы разработчика может повлиять на его удовлетворенность работой.

В разделах опроса, посвященных конструктам JCM и удовлетворенности работой, респондентам предлагалось следующее утверждение: «Пожалуйста, укажите, в какой степени ВЫ СОГЛАСНЫ с утверждениями, относящимися к вашей ТЕКУЩЕЙ работе». Поскольку мы стремились получить обратную связь от членов команды разработки, использующих как гибкие, так и негибкие методы, мы не использовали термин «гибкая разработка» в опросе до тех пор, пока респонденты не ответили на наши вопросы об использовании гибких методов. Вместо этого мы записали вопросы таким образом, чтобы члены команды, использующие и гибкие, и негибкие методы, могли их интерпретировать, и предоставили респонденту следующее утверждение: «В следующих вопросах мы хотели бы, чтобы вы задумались о том, КАК РАБОТАЕТ ВАША КОМАНДА. Когда вы видите термин «рабочий цикл», вам следует подумать о том, что ваша команда использует в качестве своего кратчайшего стандартного рабочего цикла [мы определили этот термин ранее].

****Если вы видите конкретный термин (например, «выпуск»), пожалуйста, задумайтесь об ЭТОМ термине».**

Таблица А1. Инструмент исследования

Степень использования конструкций гибкой практики		
Семибальная шкала Лайкерта («совершенно не согласен» до «мог бы согласиться или не согласиться» и «совершенно согласен») и «не знаю» восьмой вариант.		
Построить	Индикатор	Текст элемента
1. Выгорание	БД1	Наша команда использует визуальные индикаторы (диаграммы, графики и т. д.) для оценки того, насколько хорошо мы продвигаемся в ходе рабочего цикла.
	БД2	Мы используем визуальные инструменты, которые позволяют членам команды легко определить, выполняется ли работа по графику.
	БД3	Мы отмечаем выполненную работу в сравнении с запланированной на графике.
2. Итеративная доставка	ID1	В начале каждого рабочего цикла команда и владельцы бизнеса договариваются о том, что будет выполнено в течение рабочего цикла.
	ID2	Команда дает оценку объему работы, который может быть выполнен за рабочий цикл.
	ID3	Команда оценивает объем работы, который потребуется выполнить для реализации каждой функции.
	ID4	Наша команда позволяет деловым людям принимать бизнес-решения о выпусках, а техническим специалистам — технические решения о выпусках.
3. Стендап	СУ1	Каждый день команда проводит короткое совещание для обсуждения хода реализации проекта.

Таблица А1. Инструмент исследования

	SU2 Каждый день все члены команды делятся с командой тем, над чем они работают.	
	SU3 Команда ежедневно обсуждает вопросы вместе.	
4. Ретроспектива	PE1	Команда регулярно анализирует проделанную работу и ищет пути повышения эффективности работы команды.
	PE2	В конце каждого рабочего цикла команда задает себе вопрос: «Что прошло хорошо» в ходе последнего рабочего цикла.
	RE3	В конце каждого рабочего цикла команда задается вопросом: «Что можно улучшить» в следующем цикле.
5. Модульное тестирование	UT1	У нас есть отдельный набор «тестового» кода, который написан специально для тестирования «реального» кода.
	UT2	Каждый программист несет ответственность за написание модульных тестов для кода, который он пишет.
	UT3	Программисты обязаны лично запускать набор модульных тестов до тех пор, пока все они не будут выполнены успешно, прежде чем «регистрировать» изменения.
6. Непрерывная интеграция	CI1	Члены команды интегрируют изменения кода как можно скорее.
	CI2	У команды есть процесс, который автоматически перестраивает программное обеспечение несколько раз в день.
	CI3	Команда автоматически уведомляется о любых проблемах, связанных с автоматизированной компиляцией, развертыванием или тестированием кода.
7. Автоматизированная сборка	AB1	Команда использует скрипт или другой код для автоматической компиляции кода.
	AB2	Команда использует скрипт или другой код для автоматической сборки программного пакета.
	AB3	Команда использует скрипт или другой код для создания заметок о выпуске или другой документации.
8. Стандарты кодирования	CS1	Наименование и структура нашего кода являются единообразными.
	CS2	Наша команда использует стандарты для единообразного форматирования кода.
	CS3	Для успеха команды важно, чтобы весь код был отформатирован одинаково.
9. Рефакторинг	RF1	Всякий раз, когда мы видим необходимость, мы улучшаем дизайн ранее написанного нами кода.
	RF2	Каждый член команды пытается улучшить структуру кода при внесении изменений.
	RF3	Если мы находим неиспользуемый код, мы его удаляем.
10. Парное программирование	PP1	При разработке нового программного обеспечения два программиста концентрируются на написании кода.
	PP2	Мы разрабатываем наш код, используя парное программирование.
	PP3	Наш код создается двумя людьми, работающими вместе за одним компьютером.
Конструкции модели характеристик работы		
Семибальная шкала Лайкерта (от «совершенно не согласен» до «совершенно согласен»).		
Построить	Индикатор	Текст элемента
11. Обратная связь	FB1	Простое выполнение работы, требуемой должностью, дает мне массу возможностей понять, насколько хорошо я с ней справляюсь.
	FB2	После того, как я выполню задание, я знаю, хорошо ли я его выполнил.
	FB3	Работа сама по себе предоставляет вам информацию о вашей трудовой деятельности.
12. Автономность в работе	JA1	Ваша работа дает вам значительные возможности для независимости и свободы в ее выполнении.
	JA2	Ваша работа позволяет вам самостоятельно решать, как ее выполнять.
	JA3	Ваша работа дает вам возможность проявить личную инициативу и суждение при выполнении работы.

	JA4 В своей работе вы обладаете большой самостоятельностью.	
13. Разнообразие навыков	SV1 Работа требует от вас использования ряда сложных или высокоуровневых навыков.	
	CB2 Работа требует от вас выполнения множества различных действий, использования разнообразных навыков и талантов.	
	SV3 Моя работа очень разнообразна.	
14. Идентификация задачи	TI1 Работа организована таким образом, что я могу выполнить весь объем работы от начала до конца.	
	TI2 Работа дает мне возможность полностью завершить начатое дело.	
	TI3 Моя работа подразумевает выполнение «целостных» и идентифицируемых частей работы.	
15. Значимость задачи	TC1 Сама по себе работа очень значима и важна в более широком плане.	
	TC2 Результаты вашей работы, вероятно, окажут существенное влияние на жизнь и благополучие других людей.	
	TS3 Моя работа очень значима и важна.	
Удовлетворенность работой		
Семибалльная шкала Лайкерта (от «совершенно не согласен» до «совершенно согласен»).		
Построить	Индикатор	Текст элемента
16. Удовлетворенность работой	СП1	В целом я доволен своей работой.
	JS2 Я бы предпочёл другую, более идеальную работу.	
	JS3 Я удовлетворен важными аспектами своей работы.	
Отрицательная аффективность		
Пятибалльная шкала Лайкерта («совсем нет», «немного», «умеренно», «довольно сильно», «чрезвычайно»).		
Построить	Индикатор	Текст элемента
17. Отрицательная аффективность	NA1	Насколько сильно вы сейчас чувствуете страх?
	NA2	Насколько сильно вы сейчас чувствуете себя расстроенным?
	NA3	Насколько сильно вы нервничаете сейчас?
	NA4	Насколько сильно вы расстроены сейчас?
	NA5	Насколько вам стыдно сейчас?
	NA6	В какой степени вы сейчас чувствуете раздражение?

Роль	Количество респонденты	Накопленный %
Руководитель группы/менеджер проекта	88	31,0%
Архитектор	15	37,0%
Разработчик	133	89,9%
тестирование качества	18	96,8%
Бизнес-аналитик	7	99,6%
Другой	3	100.0%

Таблица А3. Сводка использования Agile-практик (VersionOne, 2011)

Классифицировать	Упражняться	% респондентов
1	Ежедневный стендап	79%
2	Планирование итераций*	74%
3	Модульное тестирование	69%
4	Ретроспективы	66%
5	Выгорание	66%
6	Планирование релиза*	65%
7	Скорость*	55%
8	Автоматизированные сборки	55%
9	Непрерывная интеграция	55%
10	Стандарты кодирования	51%
11	Рефакторинг	48%
12	Разработка через тестирование (TDD)	37%
13	Открытая рабочая зона	37%
14	Картографирование историй	34%
15	Цифровая доска задач	33%
16	Парное программирование	29%
17	Коллективное владение кодом	29%
18	Автоматизированное приемочное тестирование	25%
19	Клиент на месте	24%
20	Канбан	24%
21	Выпуск каждой функции/непрерывное развертывание	23%
22	Аналоговая панель задач	23%
23	Agile-игры	16%
24	Время цикла (в зависимости от бережливости)	11%
25	Развитие, основанное на поведении (BDD)	9%
* практики объединены в концепцию итеративной доставки (Затененные строки не включены в наш анализ)		

Приложение В: Проверка модели, тестирование смещения общего метода и

Медиализированное тестирование

Для проверки конвергентной валидности и надежности модели измерения мы следовали рекомендуемым процедурам валидации PLS, описанным Гефеном и Штраубом (2005). Для проверки конвергентной валидности мы провели бутстрап с 200 повторными выборками, а затем проанализировали внешние нагрузки модели. Конвергентная валидность подтверждается, когда все индикаторы оказывают значительную нагрузку на соответствующие латентные конструкты. В нашем случае все индикаторы продемонстрировали нагрузки, значимые на уровне 0,001 (см. таблицу В1), что свидетельствует о сильной конвергентной валидности.

Таблица В1. Т-статистика для конвергентной валидности

Построить	Подконструкция (веса 2-го порядка, Т-статистика)	Индикатор	Внешняя нагрузка	Т-статистика
Agile PM практики	Выгорание (.323, 17.882***)	BD1 выгорание	.936	92.12***
		BD2 выгорание	.943	112.70***
		BD3 выгорание	.863	35.79***
	Итеративная доставка (.330, 14.655***)	IP1 итеративная доставка	.834	28.70***
		IP2 итеративная доставка	.815	30.11***
		IP3 итеративная доставка	.786	25.37***
		IP4 итеративная доставка	.772	19.42***
	Ретроспективы (.290, 14.584***)	RE1 ретроспективы	.845	27.29***
		RE2 ретроспективы	.919	64.47***
		RE3 ретроспективы	.924	80.82***
	Ежедневный стендап (.306, 14.469***)	SU1 ежедневный стендап	.902	54.42***
		SU2 ежедневный стендап	.924	63.24***
		SU3 ежедневный стендап	.924	78.45***
Agile SDA практики	Автоматизированная сборка (.178, 9.315***)	AB1 автоматизированная сборка	.812	33.66***
		AB2 автоматизированная сборка	.868	33.35***
		AB3 автоматизированная сборка	.889	43.68***
	Непрерывная интеграция (.230, 12.038***)	CI1 продолжение интеграции	.813	27.90***
		CI2 продолжение интеграции	.882	42.44***
		CI3 продолжение интеграции	.873	44.92***
	Стандарты кодирования (.253, 12.603***)	CS1 стандарты кодирования	.893	51.53***
		CS2 стандарты кодирования	.923	67.47***
		CS3 стандарты кодирования	.876	37.01***
	Парное программирование (.210, 9.808***)	PP1 парное программирование	.888	50.42***
		PP2 парное программирование	.907	70.45***
		PP3 парное программирование	.918	66.49***
	Рефакторинг (.245, 14.198***)	RF1 рефакторинг	.804	21.82***
		RF2 рефакторинг	.902	60.78***
		RF3 рефакторинг	.878	45.39***
	Модульное тестирование (.206, 10.839***)	UT1 модульное тестирование	.719	13.14***
		UT2 модульное тестирование	.836	24.79***
		UT3 модульное тестирование	.844	31.15***
Обратная связь	Н/Д	FB1 обратная связь	.862	38.20***
		FB2 обратная связь	.880	39.36***

Таблица В1. Т-статистика для конвергентной валидности

Построить	Подконструкция (веса 2-го порядка, Т-статистика)	Индикатор	Внешняя нагрузка	Т-статистика
		FB3 обратная связь	.900	55.03***
Автономия работы	Н/Д	JA1 автономия в работе	.875	44.03***
		JA2 автономия в работе	.847	34.66***
		JA3 автономия в работе	.810	25.27***
		JA4 автономия в работе	.802	22.29***
Разнообразие навыков	Н/Д	SV1 разнообразие навыков	.824	18.44***
		SV2 разнообразие навыков	.831	19.84***
		SV3 разнообразие навыков	.871	37.73***
Идентификация задачи	Н/Д	TI1 идентификация задачи	.905	58.04***
		TI2 идентификация задачи	.928	98.22***
		TI3 идентификация задачи	.891	48.15***
Задача значение	Н/Д	TS1 значимость задачи	.817	18.37***
		TS2 значимость задачи	.833	31.90***
		TS3 значимость задачи	.920	86.62***
Отрицательно аффективность	Н/Д	NA1 отрицательное воздействие	.769	16.22***
		NA2 отрицательное воздействие	.785	21.16***
		NA3 отрицательное воздействие	.828	28.61***
		NA4 отрицательное воздействие	.789	18.34***
		NA5 негативное влияние	.714	15.12***
		NA6 негативное влияние	.725	14.12***
Удовлетворенность работой	Н/Д	СП1 удовлетворенность работой	.897	43.81***
		СП2 удовлетворенность работой	.911	59.34***
		СП3 удовлетворенность работой	.903	51.53***
*** Примечание: $p < .001$				

Другой распространённый тест конвергентной валидности — определение того, составляет ли средняя извлечённая дисперсия (AVE) не менее 0,50 (Fornell & Larcker, 1981). В нашей модели наименьшее значение AVE составило 0,592. Тесты Gefena и Штрауба (2005) и Формелла и Ларкера (1981) указывают на высокую степень конвергентной валидности.

Кроме того, для проверки надёжности элементов измерения мы использовали SmartPLS для вычисления α Кронбаха и комплексного индекса надёжности (Fornell & Larcker, 1981). Все конструкторы показали индекс надёжности, значительно превышающий пороговое значение, рекомендованное для поисковых исследований (0,60) (Nunnally, Bernstein, & Berge, 1967). Таблица В2 суммирует эти результаты.

Для оценки дискриминантной валидности мы провели два теста. Во-первых, мы изучили перекрёстные нагрузки элементов измерения на их конструкторы. В этом тесте дискриминантная валидность подтверждается, когда элемент нагружается на свой конструктор сильнее, чем на любой другой конструктор (Hair et al., 2013). Дополнительный тест предполагает, что разница в нагрузках должна превышать 0,10 (Gefen & Straub, 2005). В нашем случае все элементы прошли оба этих стандартных теста (см. таблицу В3).

Третий тест дискриминантной валидности заключается в сравнении значений AVE для каждого конструктора. В тесте дискриминантной валидности AVE квадратный корень AVE данного конструктора должен быть больше корреляции этого конструктора с любым другим конструктором в модели (Chin, 1998). Все наши конструкторы прошли этот тест, что свидетельствует о сильной дискриминантной валидности. В таблице В4 представлены значения корреляций и квадратных корней AVE.

Наконец, для оценки нашего агрегированного конструктора второго порядка мы следовали рекомендациям, предложенным Политесом и др. (2012) и Райтом, Кэмпбеллом, Тэтчером и Робертсом, (2012) при оценке формирующих факторов второго порядка, связанных с практикой Agile PM и практикой Agile SDA. На уровне первого порядка таблицы В1, В2 и В3 демонстрируют как дискриминантную, так и конвергентную валидность измерений факторов Agile PM и Agile SDA.

В соответствии с совокупной конструкцией, согласно объяснениям, которые предлагают Политес и др. (2012), соответствующие Факторы первого порядка коррелировали на значимом, но различном уровне. Для оценки валидности фактора второго порядка мы оценили, насколько значимо коррелирует каждое измерение с агрегированным конструктом. Мы обнаружили, что каждый фактор первого порядка оказывает значительное влияние на фактор второго порядка. В таблице В1 представлены нагрузки факторов первого порядка на факторы второго порядка.

Таблица В2. AVE и показатели надежности

Построить	АВЕНЮ	Композитный надежность	Кронбаха α
Гибкие конструкции PM			
Выгорание	.837	.939	.902
Итеративная доставка	.644	.878	.815
Ретроспективы	.804	.925	.877
Ежедневное совещание	.841	.941	.905
Гибкие конструкции SDA			
Автоматизированная сборка	.735	.893	.819
Непрерывная интеграция	.734	.892	.818
Стандарты кодирования	.806	.926	.879
Парное программирование	.818	.931	.889
Рефакторинг	.744	.897	.827
Модульное тестирование	.643	.843	.719
Конструкции модели характеристик работы			
Обратная связь	.776	.912	.856
Автономия работы	.696	.901	.815
Разнообразие навыков	.709	.880	.805
Идентификация задачи	.825	.934	.89
Значимость задачи	.737	.893	.824
Отрицательная аффективность	.592	.897	.861
Удовлетворенность работой	.816	.930	.888

Таблица В3. Нагрузки факторов PLS и перекрестные нагрузки

Построить		1	2	3	4	5	6				7	8	9	10	11	12	13	14	15	16	17																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																												</
-----------	--	---	---	---	---	---	---	--	--	--	---	---	---	----	----	----	----	----	----	----	----	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	----

Таблица В4. Корреляции конструкторов первого порядка

Построить 1 1. Авто.	2	3	4	5	6	7	8	9 10	11 12 13	14 15	16 17					
Построить 857																
2. Выгорание .494 .915																
3. Непрерывный инт. .635 .591 .857																
4. Кодировочный станд. .430 .488 .464 .898																
5. Обратная связь .227 .377 .335 .366 .881																
6. Итеративный план. .301 .545 .445 .416 .357 .802																
7. Работа Сб. .190 .366 .374 .312 .434 .413 .904																
8. Автономность работы .077 .149 .168 .190 .452 .268 .583 .834																
9. Отрицательное афф. -0,046 -0,182 -0,154 -0,043 -0,196 -0,158 -0,409 -0,168 0,769																
10. Парная программа. .383 .566 .516 .372 .307 .330 .246 .047 -1,107 .904																
11. Рефакторинг .405 .470 .542 .550 .367 .475 .341 .220 -.087 .465 .862																
12. Ретроспектива .458 .575 .520 .415 .362 .493 .331 .140 -.104 .416 .467 .897																
13. Разнообразие навыков .140 .279 .230 .211 .391 .303 .513 .505 .245 .126 .251 .265 .842																
14. Стойка. 0,367 0,556 0,448 0,354 0,345 0,483 0,369 0,154 -0,203 0,493 0,423 0,472 0,188 0,917																
15 Задача Сиг. .215 .338 .212 .295 .359 .311 .486 .395 -.217 .222 .290 .263 .555 .179 .858																
16 Идентификация задачи .168 .348 .267 .368 .368 .356 .514 .550 .205 .240 .386 .219 .296 .282 .317 .908																
17. Тестирование модулей .508 .580 .504 .561 .399 .415 .297 .186 .066 .364 .557 .544 .235 .404 .305 .345 .802																

Примечание: Диагональ содержит квадратный корень из AVE.

Тесты на наличие смещения общего метода

Для проверки на наличие смещения общего метода мы сначала провели однофакторный тест Хармана (Podsakoff, MacKenzie, Lee, & Podsakoff, 2003). В этом тесте все элементы включаются в невращающийся исследовательский факторный анализ, чтобы определить, проявляется ли один фактор или он объясняет большую часть дисперсии. В нашем тесте проявилось 37 факторов, самый большой из которых объяснял 36,8% дисперсии. Этот результат

было первым доказательством того, что ошибка общепринятого метода не является проблемой.

Мы также провели корреляционный тест, описанный в работе (Pavlou, Liang, & Xue, 2007). Мы обнаружили, что ни один из конструкторов не имеет чрезвычайно высокой корреляции (более 0,90). Этот результат стал вторым доказательством того, что смещение общего метода не было проблемой в нашем исследовании.

Мы выполнили тест, который предлагают Лян, Сараф, Ху и Сюэ (2007), хотя некоторые поднимали вопросы по этому тесту (Чин, Тэтчер и Райт, 2012). В этом тесте создается один фактор элемента для каждого латентного индикатора, а также один фактор «метода», содержащий все индикаторы. Затем модель запускается с этими факторами. Таблица В5 представляет результаты этого теста. Одно из них определяет смещение общего метода, исследуя статистическую значимость нагрузок конструкторов с одним элементом, а также как «субстантивных» конструкторов, так и конструктора «метод». Далее сравнивается дисперсия, объясняемая субстантивными факторами, с дисперсией, объясняемой фактором метода (Уильямс, Эдвардс и Ванденберг, 2003). Процент объясненной дисперсии равен квадрату нагрузок. Если нагрузки конструкторов метода в целом незначительны, а проценты дисперсии индикатора, объясняемые содержательными конструкторами, существенно выше, чем те, которые объясняются конструктором метода, то демонстрируется, что общее смещение метода оказывает минимальное влияние и, таким образом, не вызывает особого беспокойства.

Как показано в таблице В5, дисперсия показателей, вызванная содержательными конструкторами, была существенно выше, чем дисперсия, вызванная конструктором метода. Средняя дисперсия, вызванная содержательными конструкторами, составила 74% по сравнению с менее чем 1% для конструктора метода. Кроме того, при рассмотрении нагрузок фактора метода мы обнаружили, что подавляющее большинство факторов были незначимыми. Мы делаем вывод о незначительном влиянии общей ошибки метода.

⁶ Превосходное описание этой техники и инструкции по ее применению можно найти в работе Сипонена и Ванса (2010).

Таблица В5. Анализ смещения по общепринятому методу

Построить	Индикатор	Существенный фактор загрузки	Дисперсия объяснил	Метод факторная нагрузка	Объяснение дисперсии
1. Выгорание	БД1	.95**	.90	-.02	.000
	БД2	.91**	.83	.05	.003
	БД3	.87**	.76	-.03	.001
2. Итеративная доставка	ИП1	.82**	.67	.02	.000
	ИП2	.78**	.61	.05	.003
	ИП3	.77**	.59	.02	.000
	ИП4	.84**	.71	-.09	.008
3. Стендап	СУ1	.91**	.83	-.01	.000
	СУ2	.94**	.88	-.02	.000
	СУ3	.91**	.83	.03	.001
4. Ретроспектива	РЕ1	.80**	.64	.07	.005
	РЕ2	.93**	.86	-.02	.000
	РЕ3	.96**	.92	-.04	.002
5. Модульное тестирование	УТ1	.63**	.40	.08	.006
	УТ2	.88**	.77	-.03	.001
	УТ3	.88**	.77	-.03	.001
6. Непрерывная интеграция	СИ1	.80**	.64	.02	.000
	СИ2	.88**	.77	.005	.000
	СИ3	.89**	.79	-.02	.000
7. Автоматизированная сборка	АБ1	.71**	.50	.13*	.017
	АБ2	.92**	.85	-.06	.004
	АБ3	.94**	.88	-.05	.003
8. Стандарты кодирования	СЗ1	.89**	.79	.02	.000
	СЗ2	.92**	.85	.005	.000
	СЗ3	.88**	.77	-.02	.000
9. Рефакторинг	РФ1	.90**	.81	-.13*	.017
	РФ2	.88**	.77	.03	.001
	РФ3	.81**	.66	.08	.006
10. Парное программирование	ПП1	.93**	.86	-.05	.003
	ПП2	.89**	.79	.02	.000
	ПП3	.90**	.81	.03	.001
11. Обратная связь	ФБ1	.88**	.77	-.03	.001
	ФБ2	.89**	.79	-.01	.000
	ФБ3	.87**	.76	.04	.002
12. Автономность в работе	ЈА1	.85**	.72	.04	.002
	ЈА2	.85**	.72	-.03	.001
	ЈА3	.82**	.67	-.005	.000
	ЈА4	.82**	.67	-.01	.000
13. Разнообразие навыков	СВ1	.90**	.81	-.07*	.005
	СВ2	.93**	.86	-.121*	.015
	СВ3	.71**	.50	.20*	.040

Таблица В5. Анализ смещения по общепринятому методу

Построить	Индикатор	Существенный фактор загрузки	Дисперсия объяснил	Метод факторная нагрузка	Объяснение дисперсии
14. Идентификация задачи	ТИ1	.90**	.81	.008	.000
	ТИ2	.93**	.86	-.009	.000
	ТИ3	.89**	.79	.0009	.000
15. Значимость задачи	ТС1	.90**	.81	-.11*	.012
	ТС2	.88**	.77	-.05	.003
	ТС3	.81**	.66	.15*	.023
16. Удовлетворенность работой	СП1	.82**	.67	.10	.010
	СП2	.93**	.86	-.03	.001
	СП3	.96**	.92	-.07	.005
17. Отрицательная аффективность	НА1	.76**	.58	-.003	.000
	НА2	.80**	.64	.03	.001
	НА3	.82**	.67	-.04	.002
	НА4	.82**	.67	.03	.001
	НА5	.74**	.55	.08	.006
	НА6	.67**	.45	-.10	.010
Средний		.86	.74	.00	.00
Примечание: ** - $p < .01$; * - $p < .05$					

Об авторах

Джон Ф. Трипп — доцент кафедры информационных систем в Университете Бэйлора. До получения докторской степени он более 17 лет проработал в отрасли разработчиком программного обеспечения, руководителем проектов и ИТ-директором. Его исследования публиковались в журналах «Journal of the Association for Information Systems», «Information Systems and e-Business Management» и «Journal of Management Systems». В настоящее время он изучает такие темы, как гибкая разработка программного обеспечения, социальные сети, доверие и конфиденциальность, а также влияние технологий на лидерство, совместную работу и коммуникацию в организациях.

Синтия К. Рименшайдер — заместитель декана по исследованиям и развитию преподавательского состава, профессор кафедры информационных систем в Школе бизнеса Ханкамер при Бейлорском университете. Она также занимает должность профессора информационных систем имени Хелен Лигон. Ее публикации публиковались в журналах MIS Quarterly, Information Systems Research, Journal of Management Information Systems, Information Systems Journal, European Journal of Information Systems, IEEE Transactions on Software Engineering и других. В настоящее время она занимается исследованиями, посвященными проблемам ИТ-персонала, женщинам и меньшинствам в ИТ, а также этическим вопросам, связанным с ИТ.

использовать.

Джейсон Тэтчер — профессор информационных систем в Университете Клемсона. Его исследования посвящены факторам, влияющим на использование технологий в организациях, включая индивидуальные инновации в этой области, управление ИТ-персоналом и ИТ-стратегию организаций. Его работы публикуются в журналах MIS Quarterly, Information Systems Research, Journal of Applied Psychology и других изданиях. Он активно участвует в развитии дисциплины информационных систем. Он является избранным президентом Ассоциации информационных систем. Он является старшим редактором журналов MIS Quarterly, Decision Sciences, AIS Transaction on Human Computer Interaction и других рецензируемых изданий. Он также был членом редколлегии журналов Information Systems Research, Journal of the Association for Information Systems и IEEE Transactions on Engineering Management.

Авторские права © 2016 принадлежат Ассоциации информационных систем. Разрешение на создание цифровых или печатных копий всей данной работы или ее части для личного или учебного использования предоставляется бесплатно при условии, что копии не будут создаваться и не будут распространяться с целью получения прибыли или коммерческой выгоды, и что копии будут содержать настоящее уведомление и полную ссылку на источник на первой странице. Авторские права на компоненты данной работы, принадлежащие другим лицам, помимо Ассоциации информационных систем, должны соблюдаться. Разрешается реферирование с указанием источника. Для копирования иным способом, повторной публикации, размещения на серверах или распространения по спискам рассылки требуется предварительное специальное разрешение и/или оплата. Запросить разрешение на публикацию можно по адресу: Административный офис AIS, PO Box 2712, Атланта, Джорджия, 30301-2712, Attn: Reprints, или по электронной почте publications@aisnet.org.